Multiplayer Blackjack

Software Design Specification

Revision History

Date	Revision	Description	Author
03/05/2026	1.0	Initial Version	Michael-Eric, Curtis, Emma
03/31/2026	2.0	Added classes chapter	Curtis
04/01/2026	3.0	Added two more sequence diagrams Invalid Bet and Bet	Michael-Eric
04/03/2026	4.0	Mostly finished UML class diagrams, added Introduction chapter and Overall Description chapters Added more to the sequence diagrams, Player leaves table, Player makes a bet, and Player stands	Curtis Michael-Eric
04/05/2026	4.1	Worked on chapters	Curtis
04/06/2026	4.2	Added sequence diagrams	Emma Michael-Eric
04/07/2026	4.3	Worked on chapters, updated UML class diagrams, modified Account, ClientHandler, and GUI classes	Curtis
04/08/2026	4.4	Updated UML class diagrams, removed standAt attribute from Account class to give to Server class. Added a new Scene interface with ten new classes implementing it. Added new Used-Case Specifications and Sequence Diagrams, such as Dealer Makes A Table, Dealer Cancels Game, and Dealer Hosts Game	Curtis Michael-Eric
04/09/2026	4.5	Finished UML class diagrams, modified Scene classes, writing more for chapters, added new use cases to Use Case Diagram. SDS ready for upload.	Curtis
04/09/2026	4.6	Finished up account-related Use-Case Specifications and Sequence Diagrams	Emma
05/04/2026	5.0	Modifying classes based on details added during implementation phase	Curtis
05/05/2026	5.1	Finalize changes made to classes and in design explanations Modified UML diagrams and use cases to reflect changes (library dependency graph removed for being requiring too much work to maintain)	Curtis

05/05/26	5.2	Updated Ch 3 Client description	Emma

Table of Contents

1. Introduction

1.1. Scope

The SDS covers the design choices to be used in the implementation of the Multiplayer Blackjack product. The next chapter describes the general structure of the product and the applications that it consists of. The third chapter describes the classes that will be implemented and their attributes and methods. Finally, the remainder of the SDS will consist of diagrams that show the use cases of the product, the relationships between the product's classes, and how the product accomplishes its goals for the user.

For information on what the goals of the product are, refer to the SRS.

For information on how to test the product, refer to the Test Plan.

1.2. Overview

The Multiplayer Blackjack product simulates the experience of walking into a casino and playing blackjack. Players join a table with a dealer and bet from their balance. Players can hit or stand to get their score as close to 21 as possible without going over. If a player's score is better than the dealer's, they win their bets, but if the dealer has a better score, the player loses their bets.

A GUI will facilitate user interaction with the Multiplayer Blackjack product. Users register accounts that are saved on the server side. Users can then join a table or, if they're a player, can check and adjust their balance. At a table, users play blackjack with the aforementioned rules.

2. Product Details

2.1. Product architecture

The entire product is split into two applications that share a total of eighteen classes. These classes are the Client, Server, Table, Account, Card, CardDeck, Message, ClientHandler, GUI, and BlackjackApp. One application, MBClient, will have the Client, GUI, and Blackjack App classes plus eight additional classes inheriting from the Scene interface. These classes are DealerMainMenuScene, DealerTableScene, PlayerMainMenuScene, PlayerTableScene, BalanceScene, OpenScene, LoginScene, and RegisterScene. The other application, MBServer, will have the Server, Table, Account, Card, CardDeck, and ClientHandler classes. Both applications share the Message class.

In the SRS, six modules were envisioned to cover the breadth of Multiplayer Blackjacks functionality. The Server module is represented through every class in MBServer, including the Message class. The Client module is represented through every class in MBClient, including the Message class. The Player module and the Dealer module are represented through every class in MBClient plus the Message, Server, and Account classes. The Table module is represented by the Table, the ClientHandler, and the Server classes. Finally, the Game module includes all classes from the Table module plus Account, Card, and CardDeck classes.

2.2. Dependencies

2.2.1. Java.io.*
- 2.2.1.1. Allows Messages to be sent through the internet
- 2.2.1.2. Allows for reading and writing to files

2.2.2. java.net.*
- 2.2.2.1. Enables client-server communications

2.2.3. java.awt.*
- 2.2.3.1. For building GUI

2.2.4. java.awt.event.*
- 2.2.4.1. For building GUI

2.2.5. java.swing.*
- 2.2.5.1. For building GUI

2.2.6. java.util.*
- 2.2.6.1. Enables vectors and lists
- 2.2.6.2. Enables executors that utilize timer

2.3. Constraints

2.3.1. TCP/IP will be used for communication between MBClient and MBServer.
2.3.2. Java will be the only language used to create the Multiplayer Blackjack product.
2.3.3. No databases will be used to store data; text files will be used instead.

2.4. External features

2.4.1. Appropriate dealer credentials are to be defined by the clientele.
2.4.2. Banking services are to be provided by a third party specializing in finance.

3. Classes

3.1. Client

3.1.1. Description: The Client class would act as a communicator between the user and the server.

3.1.2. Attributes:

3.1.2.1. private int port: *Port number of server to connect to*

3.1.2.2. private String host: *IP address of the user*

3.1.2.3. private Socket socket: *Socket used to connect to server*

3.1.2.4. private OutputStream outputStream: *Stream used to initialize objectOutputStream*

3.1.2.5. private InputStream inputStream: *Stream used to initialize objectInputStream*

3.1.2.6. private ObjectOutputStream objectOutputStream: *Stream used to send Messages to server*

3.1.2.7. private ObjectInputStream objectInputStream: *Stream used to receive Messages from server*

3.1.2.8. private boolean loggedIn: *record of whether the user is logged in to the Multiplayer Blackjack server*

3.1.2.9. private boolean[] seatedPlayers: *record of how many players are seated at the table during an active game session*

3.1.2.10. private String IPAddress: *record of user's IP address*

3.1.2.11. private String username: *record of user's username*

3.1.2.12. private Vector<Message> messageLog: *log of messages sent and received during client lifetime*

3.1.3. Methods:

3.1.3.1. public Client(): *Default constructor*

3.1.3.2. public Client(int port, String host): *Constructor that initializes port and host*

3.1.3.3. public void Connect(): *Establishes connection with Server*

3.1.3.4. public boolean verifyConnection(): *Verify successful message sent to and from Server to confirm working object streams*

3.1.3.5. public boolean requestLogIn(String username, String password, String credentials): *LogIn message sent to Server containing username, password, credentials; LogIn message received indicating Success or Fail; true returned if successful, false returned if not successful*

3.1.3.6. public boolean requestRegistry(String username, String password, String credentials): *Register message sent to Server containing username, password, credentials; Register message received indicating Success or Fail; true returned if successful, false returned if not successful*

3.1.3.7. public boolean requestLogOut(): *LogOut message sent to server; true returned if logout successful, false returned if not successful*

3.1.3.8. public double requestBalanceView(): *BalanceRequest message sent to server; balance returned*

3.1.3.9. public double requestBalanceDeposit(): *DepositRequest message sent to server containing deposit amount; New balance returned*

3.1.3.10. public double requestBalanceWithdraw(): *WithdrawRequest message sent to server containing withdraw amount; New balance returned*

3.1.3.11. public boolean joinTable(): *TableJoin message sent to server; true returned if successful, false returned if not successful*

3.1.3.12. public boolean leaveTable(): *TableLeave message sent to server; true returned if successful, false returned if not successful*
3.1.3.13. public boolean chooseBet(double bet): *TableLeave message sent to server; true returned if successful, false returned if not successful*
3.1.3.14. public void hit(): *Hit message sent to server*
3.1.3.15. public void stand(): *Stand message sent to server*
3.1.3.16. public int getGameUsers(): *RenderPlayers message sent to server; number of players at table returned*
3.1.3.17. private void send(Message msg): *Passed message sent to server*
3.1.3.18. private Message receive(): *Message received from server and returned from function*

3.2. Server

3.2.1. Description: The Server class holds information on users (i.e. username, cards in hand, etc.) and modifies them based on client communications.
3.2.2. Attributes:
3.2.2.1. private static Vector<Table> availableTables: *Record of tables available for users to join*
3.2.2.2. private static Vector<Account> accountRegistry: *Record of registered user accounts*
3.2.2.3. private static ServerSocket socket: *IP address needed to connect to server*
3.2.2.4. private static Vector<Message> messageLog: *Log of messages sent and received during server lifetime*
3.2.2.5. private static int standAt: *The score the dealer stands at*
3.2.3. Methods:
3.2.3.1. public static void main(String[] args): *Loads account registry information from text files and listens for clients wishing to connect to server*

3.3. Table

3.3.1. Description: The Table class represents a game table and manages the game.
3.3.2. Attributes:
3.3.2.1. private ClientHandler[] players: *Record of players currently playing at the table*
3.3.2.2. private ClientHandler dealer: *Record of the dealer currently playing at the table*
3.3.2.3. private CardDeck shoe: *Record of the deck(s) of cards used*
3.3.2.4. private int timer: *Timer used to control flow of table activities*
3.3.2.5. private boolean dealerLeave: *Queue for the dealer to leave the table*
3.3.2.6. private boolean gameActive: *Record of whether users are in the middle of a game*
3.3.2.7. private int playersTurn: *Record of whose turn it currently is*
3.3.2.8. private Vector<Card> cardsDrawn: *Record of cards drawn in a game*
3.3.3. Methods:
3.3.3.1. public Table(): *Construct table*
3.3.3.2. public void startGame(): *Start a game of blackjack at the table*
3.3.3.3. public void nextTurn(): *Get the server to request an action from the user whose turn it is*
3.3.3.4. public void endGame(): *End the game of blackjack by paying bets and resetting shoe, timer, and cardsDrawn*
3.3.3.5. public Card drawCard(): *Draw a card from the shoe for the current user's turn and track the card drawn*

3.3.3.6. public int getPlayerCount(): *Return number of players at the table*
3.3.3.7. public boolean hasDealer(): *Return if table has dealer playing at it*
3.3.3.8. public void addUserToTable(ClientHandler user): *Add a player or a dealer to the table*
3.3.3.9. public void queueLeave(ClientHandler user): *Queue user to leave table; players leave instantly*
3.3.3.10. private void removeUserFromTable(ClientHandler user): *Remove a player or dealer from the table*
3.3.3.11. public boolean getVacancies(): *Return boolean list indicating which entries in the players attribute are null*
3.3.3.12. public Card getCardsDrawn(): *Return record of cards drawn during a game of blackjack*
3.3.3.13. public void playerStoodOrBust(): *Get information on whether a player can act on their turn*
3.3.3.14. public void dealerCancelledGame(): *Remove all users from the table, refunding any active bets*
3.3.3.15. public Vector<Card>[] getAllUsersHands(): *Returns array of hands each player has at the table for rendering purpose*

3.4. Account

3.4.1. Description: The Account class is a record for user information that is kept server-side for safety.
3.4.2. Attributes:
3.4.2.1. private String Username: *Record of user username*
3.4.2.2. private String Password: *Record of user password*
3.4.2.3. private boolean Credentials: *Record of if user has dealer credentials*
3.4.2.4. private double balance: *Record of player balance*
3.4.2.5. private int timeOut: *Record of time remaining until player can play again*
3.4.2.6. private Vector<Card> cards: *Record of cards in hand during a blackjack game*
3.4.2.7. private double activeBet: *Record of amount betted during blackjack game*
3.4.2.8. private boolean signedIn: *Record of if account is currently being used*
3.4.3. Methods:
3.4.3.1. public void Account(String username, String password, String credentials): *Constructs account with identification for registration*
3.4.3.2. public void Account(String username, String password, String credentials, double balance, int timeOut): *Constructs account with identification plus balance and time out duration for server initialization*
3.4.3.3. public boolean modifyBalance(double balance): *Subtracts or adds to balance, returning if function was able to accomplish this*
3.4.3.4. public int setTimeOut(int timer): *Sets time out timer and has it tick down in the background*
3.4.3.5. public boolean isTimedOut(): *Returns if player is in time out*
3.4.3.6. public boolean setBet(double bet): *Sets player's bet, returning if sufficient funds are available for bet*
3.4.3.7. public void resetCardsAndBet(): *Empties cards and activeBet attributes*
3.4.3.8. public void resetCardsAndBet(boolean payOut): *Empties cards and activeBet attributes and payouts*
3.4.3.9. public void receiveCard(Card card): *adds card to hand*
3.4.3.10. public Card getCards(): *Return copy of cards in hand*

3.4.3.11. public boolean validate(String username, String password, String credentials): *Validates log in attempt by comparing arguments to corresponding attributes and seeing if account is already logged in*
3.4.3.12. public double getBalance(): *Returns balance*
3.4.3.13. public boolean SignOut(): *Signs user out of account, returns if account was logged in*
3.4.3.14. public int getTimeOut(): *Returns remaining time out*
3.4.3.15. public String getUsername(): *Returns username*
3.4.3.16. public boolean getCredentials(): *Returns credentials*

3.5. Card

3.5.1. Description: The Card class represents a playing card used for playing blackjack.
3.5.2. Attributes:
3.5.2.1. private String rank: *Card's rank*
3.5.2.2. private String suit: *Card's suit*
3.5.3. Methods:
3.5.3.1. public Card(String rank, String suit): *Construct a card with a rank and suit*
3.5.3.2. public void getRank(): *Returns rank*
3.5.3.3. public void getSuit(): *Returns suit*
3.5.3.4. public void toString(): *Returns string representation card*

3.6. CardDeck

3.6.1. Description: The CardDeck class represents a set of decks of cards that are shuffled together.
3.6.2. Attributes:
3.6.2.1. private List<Card> cards: *Record of cards in CardDeck*
3.6.2.2. private int maxDecks: *number of decks that compose CardDeck*
3.6.3. Methods:
3.6.3.1. public CardDeck(int maxDecks): *Construct CardDeck with the number of decks it holds*
3.6.3.2. public void shuffle(): *Shuffle cards held in decks*
3.6.3.3. public Card pullCard(): *Remove a card from CardDeck and return it*
3.6.3.4. public void reset(): *replace cards held in CardDeck with copies of standard 52 card decks equal to maxDecks*
3.6.3.5. public void addMaxDecks(): *Increment number of decks CardDeck can hold*

3.7. Message

3.7.1. Description: The Message class is a serializable record of information sent between client and server to request an action.
3.7.2. Attributes:
3.7.2.1. protected final MessageType type: *Type of message*
3.7.2.2. protected final MessageStatus status: *Success status of message*
3.7.2.3. protected final String text: *Information recorded in message to be deciphered based on message type*
3.7.3. Methods:
3.7.3.1. public Message(): *Constructor to instantiate dummy messages*
3.7.3.2. public Message(MessageType type, MessageStatus status): *Constructor to instantiate messages that don't need to send additional information*

3.7.3.3. public Message(MessageType type, MessageStatus status, String text): *Constructor to instantiate messages that need to send additional information*
3.7.3.4. public MessageType getType(): *Returns type of message*
3.7.3.5. public MessageStatus getStatus(): *Returns status of message*
3.7.3.6. public String getText(): *Returns information included in message*

3.8. ClientHander

3.8.1. Description: The ClientHandler class allows multiple users to join a server. It is an inner class of the Server class.
3.8.2. Attributes:
3.8.2.1. private Socket client: *Client's socket*
3.8.2.2. private Account account: *Account of logged in user*
3.8.2.3. private boolean stoodOrBust: *Record of whether the user cannot take their turn in a blackjack game*
3.8.2.4. private Table seatedAt: *Table user is seated at for blackjack games*
3.8.2.5. private ObjectOutputStream out: *Stream used to send Messages to client*
3.8.2.6. private ObjectInputStream in: *Stream used to receive Messages from client*
3.8.3. Methods:
3.8.3.1. public ClientHandler(): *Construct ClientHandler without a client to handle*
3.8.3.2. public ClientHandler(Socket client): *Construct ClientHandler with client to handle*
3.8.3.3. public void run(): *Automatically runs on thread creation, causing ClientHandler to listen to client messages during its lifetime*
3.8.3.4. public synchronized void lookForTable(): *Look for a table for users to join, prioritizing less populated tables for players.*
3.8.3.5. public synchronized void makeTable(): *Add a table for users to join*
3.8.3.6. public void timeOut(): *Time a player out from playing blackjack for 300 seconds*
3.8.3.7. public synchronized boolean register(String username, String password, String credentials): *Register an account for a user*
3.8.3.8. public boolean logIn(String username, String password, String credentials): *Log into an account for a user*
3.8.3.9. public double getPlayerBalance(): *Returns player balance for a user*
3.8.3.10. public double chargePlayerBalance(double currency): *Subtracts from player balance, returning resulting balance*
3.8.3.11. public double addToPlayerBalance(double currency): *Adds to player balance, returning resulting balance*
3.8.3.12. public void informClientOfError(ErrorType errorType): *Sends error message to client*
3.8.3.13. public void askForBets(): *Tells client to ask the player for a bet, repeating until valid bet made*
3.8.3.14. public void removeFromTable(): *Request table to be removed*
3.8.3.15. public void askForAction(): *Tells client to ask the user to hit or stand*
3.8.3.16. public void hitRequest(): *While in a game, hit*
3.8.3.17. public void standRequest(): *While in a game, stand*
3.8.3.18. public void addCard(Card card): *Adds card to account hand*
3.8.3.19. public void restartGame(): *Resets game-specific attributes in account*
3.8.3.20. public void restartGame(boolean payOut): *Resets game-specific attributes in account, paying out user for tying or winning against dealer*

3.8.3.21. public void getGameUsers(): *Tells client what vacancies exist at the table they're seated at*
3.8.3.22. public void getGameCards(): *Tells client what cards each player has (minus the hole)*
3.8.3.23. public int checkRanks(): *Check rank of cards in hand to see blackjack score, returning value of hand*
3.8.3.24. public void save(): *Saves account information*
3.8.3.25. public void getStoodOrBust(): *Returns whether the player is able to take their turn*
3.8.3.26. public synchronized void collectMessage(Message message): *Collects message and adds it to the messageLog*
3.8.3.27. public void writeMessage(Message message): *Sends message to client and records it in messageLog*
3.8.3.28. public boolean isDealer(): *Returns if user is a dealer*
3.8.3.29. public void cancelGame(): *If user is a dealer, remove everyone seated at the table, including caller*
3.8.3.30. public Vector<Card> getHand(): *Returns the hand the user has*

3.9. GUI

3.9.1. Description: The GUI class is the UI created for users by the client. It is an inner class of the Client class.
3.9.2. Attributes:
3.9.2.1. private Client client: *Client who is GUI services*
3.9.2.2. private Scene currentScene: *Current backdrop of the GUI*
3.9.2.3. private JFrame mainFrame: *GUI frame used render GUI*
3.9.3. Methods:
3.9.3.1. public GUI(Client client): *Construct GUI and get client to service*
3.9.3.2. public void run(): *Create mainFrame to render GUI for user*
3.9.3.3. public void setScene(Scene scene): *Change backdrop of GUI*

3.10. DealerMainMenuScene

3.10.1. Description: The DealerMainMenuScene class contains GUI elements for the GUI class for the main menu for dealers. Implements the Scene interface.
3.10.2. Methods:
3.10.2.1. public DealerMainMenuScene(): *Construct scene*
3.10.2.2. public void getPanel(): *Return panel used to build scene*

3.11. DealerTableScene

3.11.1. Description: The DealerTableScene class contains GUI elements for the GUI class for the table for dealers. Implements the Scene interface.
3.11.2. Methods:
3.11.2.1. public DealerMainMenuScene(): *Construct scene*
3.11.2.2. public void getPanel(): *Return panel used to build scene*

3.12. PlayerMainMenuScene

3.12.1. Description: The PlayerMainMenuScene class contains GUI elements for the GUI class for the main menu for players. Implements the Scene interface.
3.12.2. Methods:
3.12.2.1. public DealerMainMenuScene(): *Construct scene*
3.12.2.2. public void getPanel(): *Return panel used to build scene*

3.13. PlayerTableScene

3.13.1. Description: The PlayerTableScene class contains GUI elements for the GUI class for the table for players. Implements the Scene interface.

3.13.2. Methods:

3.13.2.1. public DealerMainMenuScene(): *Construct scene*

3.13.2.2. public void getPanel(): *Return panel used to build scene*

3.14. BalanceScene

3.14.1. Description: The BalanceScene class contains GUI elements for the GUI class for viewing a player's balance. Implements the Scene interface.

3.14.2. Methods:

3.14.2.1. public DealerMainMenuScene(): *Construct scene*

3.14.2.2. public void getPanel(): *Return panel used to build scene*

3.15. OpenScene

3.15.1. Description: The OpenScene class contains GUI elements for the GUI class for opening the Client application. Implements the Scene interface.

3.15.2. Methods:

3.15.2.1. public DealerMainMenuScene(): *Construct scene*

3.15.2.2. public void getPanel(): *Return panel used to build scene*

3.16. LoginScene

3.16.1. Description: The LoginScene class contains GUI elements for the GUI class for the log in screen for users. Implements the Scene interface.

3.16.2. Methods:

3.16.2.1. public DealerMainMenuScene(): *Construct scene*

3.16.2.2. public void getPanel(): *Return panel used to build scene*

3.17. RegisterScene

3.17.1. Description: The DealerRegisterScene class contains GUI elements for the GUI class for the register screen for both players and dealers. Implements the Scene interface.

3.17.2. Methods:

3.17.2.1. public DealerMainMenuScene(): *Construct scene*

3.17.2.2. public void getPanel(): *Return panel used to build scene*

3.18. BlackjackApp:

3.18.1. Description: The BlackjackApp class is driver that loads a Client object and a GUI object

3.18.2. Methods:

3.18.2.1. public static void main(String[] args): *Initializes a Client and a GUI object, verifies the client's connection to the server, and run the GUI*

r

Use Case Specifications

Use Case ID: USER-01
Use Case Name: Register Player
Relevant Requirements:
- 3.1.4.1
- 3.1.5.1

Primary Actor: Player
Pre-conditions:
- There is no account currently logged in on the user's system.

Post-conditions:
- The user has created a player account.
- The user is logged into the new player account.

Basic Flow or Main Scenario:
1. The GUI prompts for an action on OpenScene(sign in, register as player, or register as dealer).
2. The user chooses to register as a player.
3. GUI is updated to PlayerRegisterScene.
4. The GUI prompts for a username and password to create an account with.
5. The user enters a username and password.
6. The GUI validates the username and password.
7. Message of type Register sent from Client to Server holding username, password and account credentials.
8. Client handler takes username, password, and credentials and makes a new account, returning a message of type Registered indicating successful account registry and log in to Client.
9. Client GUI is updated to PlayerMainMenuScene.

Extensions or Alternate Flows:
Extension 1: Username is Invalid
- 5.1. The system informs the user that the username is invalid (not available).
- 5.2. Process returns to step 3.

Extension 2: Password is Invalid
- 5.1. The system informs the user that the password is invalid (doesn't meet requirements).
- 5.2. Process returns to step 3.

Exceptions:
Exception 1: Invalid Input
Related Use Cases: N/A

Use Case ID: USER-02
Use Case Name: Register Dealer
Relevant Requirements:
- 3.1.4.1
- 3.1.5.1

Primary Actor: Dealer
Pre-conditions:
- There is no account currently logged in on the user's system.

Post-conditions:
- The user has created a dealer account.
- The user is logged into the new dealer account.

Basic Flow or Main Scenario:
1. The GUI prompts for an action on OpenScene(sign in, register as player, or register as dealer).
2. The user chooses to register as a dealer.
3. GUI is updated to DealerRegisterScene.
4. The GUI prompts for a username and password to create an account with.
5. The user enters a username and password.
6. The GUI validates the username and password.
7. Message of type Register sent from Client to Server holding username, password and account credentials.
8. Client handler takes username, password, and credentials and makes a new account, returning a message of type Registered indicating successful account registry and log in to Client.
9. Client GUI is updated to DealerMainMenuScene.

Extensions or Alternate Flows:
Extension 1: Username is Invalid
- 5.3. The system informs the user that the username is invalid (not available).
- 5.4. Process returns to step 3.

Extension 2: Password is Invalid
- 5.3. The system informs the user that the password is invalid (doesn't meet requirements).
- 5.4. Process returns to step 3.

Exceptions:
Exception 1: Invalid Input
Related Use Cases: N/A

Use Case ID: USER-03
Use Case Name: Log in
Relevant Requirements:
- 3.1.4.2
- 3.1.5.2

Primary Actor: User (Player or Dealer)
Pre-conditions:
- There is no account currently logged in on the user's system.
- The account exists.

Post-conditions:
- The user is logged in on their system.

Basic Flow or Main Scenario:
1. The GUI prompts for an action on OpenScene (sign in, register as player, or register as dealer).
2. The user chooses to sign in.
3. GUI is updated to LogInScene.
4. The GUI prompts for a username and password.
5. The user enters a username and password.
6. The GUI validates the username and password.
7. Message of type LogIn sent from Client to Server holding the entered username and password, and account credentials.
8. ClientHandler takes username, password, and credentials and attempts logIn.
9. Account returns true boolean indicating successful login.
10. ClientHandler returns a message of type SuccessfulLogIn to Client.
11. Client GUI is updated to PlayerMainMenuScene.

Extensions or Alternate Flows:
Alternative 1: Login Failure
9. Account returns false boolean indicating failed login.
10. ClientHandler returns a message of type Error with ErrorType of either InvalidUsernameOrPassword or InvalidCredentials to Client.
11. The GUI notifies the user of the failed login.
12. The GUI prompts for a username and password.
13. Repeat from step 6.

Exceptions:
Exception 1: Input Invalid
Related Use Cases:
- USER-01: Register Player
- USER-02: Register Dealer

Use Case ID: USER-04
Use Case Name: Log Out
Relevant Requirements:
- 3.1.4.2
- 3.1.5.2

Primary Actor: User (Player or Dealer)
Pre-conditions:
- The account exists. and is currently logged in.

Post-conditions:
- The user is logged out.

Basic Flow or Main Scenario:
1. The system prompts for an action (sign in, register as player, or register as dealer).
2. The user chooses to log out.
3. Client sends message of type LogOut to Server, indicating signed in user wants to log out.
4. ClientHandler calls logOut in account which updates signedIn boolean attribute to false.
5. Control returns to Client.
6. Client GUI is updated to OpenScene.

Extensions or Alternate Flows:
N/A
Exceptions:
Exception 1: Input Invalid
Related Use Cases:
- USER-03: Log In

Use Case ID: PLAYER-01
Use Case Name: Check Balance
Relevant Requirements:
- 3.1.5.5
- 3.2.5

Primary Actor: Player
Pre-conditions:
- The player is logged in.

Post-conditions:
- The player's current balance is displayed to the player.

Basic Flow or Main Scenario:
1. PlayerMainMenuScene prompts action from the player(view account balance, join blackjack table, or log out).
2. The player chooses to view balance.
3. The Client requests the current balance attached to the signed in player's account from Server via message of type BalanceRequest.
4. ClientHandler fetches account balance.
5. Balance returned to Client via a message of type BalanceView.
6. GUI updates to BalanceScene, displaying the player's current balance.

Extensions or Alternate Flows: N/A
Exceptions: N/A
Related Use Cases:
- USER-03: Log in

Use Case ID: PLAYER-02
Use Case Name: Deposit
Relevant Requirements:
- 3.1.5.6
- 3.2.5

Primary Actor: Player
Pre-conditions:
- The player is logged in.

Post-conditions:
- The player's current balance is updated including the new deposit.

Basic Flow or Main Scenario:
1. PlayerMainMenuScene prompts action from the player(view account balance, join blackjack table, or log out).
2. The player chooses to view account balance.
3. The Client requests the current balance attached to the signed in player's account from Server via message of type BalanceRequest.
4. ClientHandler fetches account balance.
5. Balance returned to Client via a message of type BalanceView.
6. GUI updates to BalanceScene, displaying the player's current balance.
7. GUI prompts action from the player(deposit, withdraw, back).
8. The Player chooses to deposit.
9. GUI prompts deposit amount from player.
10. The player enters a deposit amount.
11. GUI validates the entered deposit amount.
12. The Client requests a balance deposit from the Server via a message of type DepositRequest.
13. ClientHandler adds the given amount from the player's account balance.
14. New balance returned to Client via a message of type Deposit.
15. GUI updates BalanceView to display new balance.

Extensions or Alternate Flows:
Extension 1: Invalid deposit input
- 11.1. The system informs the user that the input is invalid.
- 11.2. Process returns to step 5.

Exceptions:
Exception 1: Input Invalid
Related Use Cases:
- USER-03: Log in

Use Case ID: PLAYER-03
Use Case Name: Withdraw
Relevant Requirements:
- 3.1.5.6
- 3.2.5

Primary Actor: Player
Pre-conditions:
- The player is logged in.

Post-conditions:
- The player's current balance is updated minus withdrawal.

Basic Flow or Main Scenario:
1. PlayerMainMenuScene prompts action from the player(view account balance, join blackjack table, or log out).
2. The player chooses to view account balance.
3. The Client requests the current balance attached to the signed in player's account from Server via message of type BalanceRequest.
4. ClientHandler fetches account balance.
5. Balance returned to Client via a message of type BalanceView.
6. GUI updates to BalanceScene, displaying the player's current balance.
7. GUI prompts action from the player(deposit, withdraw, back).
8. The Player chooses to withdraw.
9. GUI prompts deposit amount from player.
10. The player enters a withdrawal amount.
11. GUI validates the entered deposit amount (amount<=current balance).
12. The Client requests a balance withdrawal from the Server via a message of type WithdrawRequest.
13. ClientHandler removes the given amount from the player's account balance.
14. New balance returned to Client via a message of type Withdraw.
15. GUI updates BalanceView to display new balance.

Extensions or Alternate Flows:
Extension 1: Invalid deposit input
- 11.3. The system informs the user that the input is invalid.
- 11.4. Process returns to step 5.

Exceptions:
Exception 1: Input Invalid
Related Use Cases:
- USER-03: Log in

Use Case ID: PLAYER-04
Use Case Name: “Join table”
Relevant Requirements: 3.1.2, 3.1.3, 3.1.7
Primary Actor: Player or dealer
Pre-conditions: The user, whether it be a player or dealer, is logged into the program. The user also does not have a time out.
Post-conditions: The user is at a table. A table may be opened.
Basic Flow or Main Scenario:

1.The player is logged into the Multiplayer Blackjack program;
2. The player requests to play blackjack from the client UI;
3. The client requests the server find an available table;
4. The server finds an available table;
5. The player joins that table.

Extensions or Alternate Flows:
Extension 1: no table available

1.The player is logged into the Multiplayer Blackjack program;
2. The player requests to play blackjack from the client UI;
3. The client requests the server find an available table;
4. The server finds no available table, so it creates a new table;
5. The player joins the new table.

Alternate Flow 1: Dealer case

1.The dealer is logged into the Multiplayer Blackjack program;
2. The dealer requests to play blackjack from the client UI;
3. The client requests the server find an available table;
4. The server finds an available table;
5. The dealer joins that table.

Extension 2: Dealer case, no table available

1.The dealer is logged into the Multiplayer Blackjack program;
2. The dealer requests to play blackjack from the client UI;
3. The client requests the server find an available table;
4. The server finds no available table, so it creates a new table;
5. The player joins the new table.

Extension 3: Player timed out

1.The player is logged into the Multiplayer Blackjack program;
2. The player requests to play blackjack from the client UI;
3. The client requests the server find an available table;
4. The server informs the client that the player is timed out for some time;
5. The client UI displays that the player is timed out and how long until they are free to join again.

Exceptions: The player is timed out
Related Use Cases: USER-03: Log in, PLAYER-05: Bet

Use Case ID: PLAYER-05
Use Case Name: “Bet”
Relevant Requirements: 3.1.2, 3.1.3, 3.1.5, 3.1.6
Primary Actor: Player
Pre-conditions: The player is at a table and is currently active in a blackjack game.
Post-conditions: The player has placed a bet, which is subtracted from their balance.
Basic Flow or Main Scenario:

1.The player is logged into the Multiplayer Blackjack program;
2.The player joins a table;
3. A game of blackjack begins;
4. The player is prompted by the server to place a bet;
5. The player chooses the chips they wish to bet with through the UI;
6. The server checks the player’s balance;
7. The player can afford the bet;
8. The bet is placed, with currency subtracted from the balance equal to the worth of the chips.

Extensions or Alternate Flows:
Extension 1: Invalid bet

1.The player is logged into the Multiplayer Blackjack program;
2.The player joins a table;
3. A game of blackjack begins;
4. The player is prompted by the server to place a bet;
5. The player chooses the chips they wish to bet with through the UI;
6. The server checks the player’s balance;
7. The player cannot afford the bet;
8. The server tells the client that the player cannot place the bet;
9. The process repeats from step 4;
10. If no bet is placed in time, a bet of the lowest amount is made automatically.

Exceptions: The player has insufficient balance or runs out of time.
Related Use Cases:

- USER-03: Log in
- USER-04: Join table
- USER-06: Hit
- USER-07: Stand

Use Case ID: USER-05
Use Case Name: “Leave table”
Relevant Requirements: 3.1.7
Primary Actor: Player or dealer
Pre-conditions: The user is at a table.
Post-conditions: The user is returned to the starting menu or will be returned after a game concludes.
Basic Flow or Main Scenario:

1.The player is logged into the Multiplayer Blackjack program;
2.The player is at a table;
3. The player requests from the client UI to leave the table;
4. The server removes the user from the table; 5. The player is back at the starting menu.

Extensions or Alternate Flows:
Alternate flow 1: Dealer case, game active

1. The dealer is logged into the Multiplayer Blackjack program;
2.The dealer is at a table;
3. The dealer requests from the client UI to leave the table;
4. There is a blackjack game active;
5. The server keeps a record that the dealer wants to leave;
6. After the game ends, the dealer automatically leaves the table;
7. The dealer is back at the starting menu.

Extension 1: Dealer case, no game active

1. The dealer is logged into the Multiplayer Blackjack program;
2. The dealer is at a table;
3. The dealer requests from the client UI to leave the table;
4. There is no blackjack game active;
5. The server removes the dealer from the table;
6. The dealer is back at the starting menu.

Exceptions: N/A
Related Use Cases:

- USER-03: Log in
- USER-04: Join table

Use Case ID: USER-06
Use Case Name: “Hit”
Relevant Requirements: 3.1.2, 3.1.3, 3.1.4, 3.1.5, 3.1.7
Primary Actor: Player/Dealer
Pre-conditions: User is at a table, they are not busted, and it is currently their turn
Post-conditions: The user receives an additional card; if the total exceeds 21, the player busts And the turn ends
Basic Flow or Main Scenario:

1. The user requests an additional card from the Client UI.
2. The Client sends the “Hit” request to the server.
3. The server tells the Table class to execute the pullCard method from the CardDeck.
4. The system adds the card to the player’s hand and updates the hand total.
5. The system displays the new card and updates the total to the player.

Related Use Cases:

- USER-04: Join table
- USER-07: Stand

Use Case ID: USER-07
Use Case Name: “Stand”
Relevant Requirements: 3.1.3, 3.1.4, 3.1.5, 3.1.7
Primary Actor: Player/Dealer
Pre-conditions: The player is at a table, and it’s currently their turn
Post-conditions: The user is disconnected from the server and the application closes
Basic Flow or Main Scenario:

1. The user selects the “Stand” option from the Client UI.
2. The Client sends a standRequest.
3. The Server signals the Table class to trigger the nextTurn method.
4. The system locks the player’s UI for the remainder of the round.

Related Use Cases:

- USER-04: Join table
- USER-06: Hit

Use Case ID: USER-08
Use Case Name: “Exit Program”
Relevant Requirements: 3.1.3, 3.1.4, 3.1.5, 3.1.7
Primary Actor: Player/Dealer
Pre-conditions: The program is currently running.
Post-conditions: The user is disconnected from the server, and the application closes
Basic Flow or Main Scenario:

1. The user selects the “Exit” or “Close” option from the Client UI.
2. The Client sends a leaveRequest to the Server.
3. The Server removes the user from any active tables via the removeUserFromTable method.
4. The Server terminates the connection session.
5. The Client application closes.

Related Use Cases:

- USER-03: Log in
- USER-05: Leave table

Use Case ID: DEALER-01
Use Case Name: “Cancels Game”
Relevant Requirements: 3.1.7
Primary Actor: Dealer
Pre-conditions: The user is at a table.
Post-conditions: The Dealer and Players return to the starting menu or will be returned and the table’s game will be cancelled and removed from the server
Basic Flow or Main Scenario:

1. The dealer selects the option to cancel the game from the Client UI
2. The Client sends a cancellation request Message to the Server
3. The Server receives a request and calls the removeFromTable() method from the ClientHandler
4. The Server triggers the removeUserFromTable(ClientHandler user) method for the dealer and all seated players
5. The server updates the Client UI to reflect the SceneType change back to the main menu for all disconnected users

Related Use Cases:

- USER-03: Log in
- USER-05: Leave Table

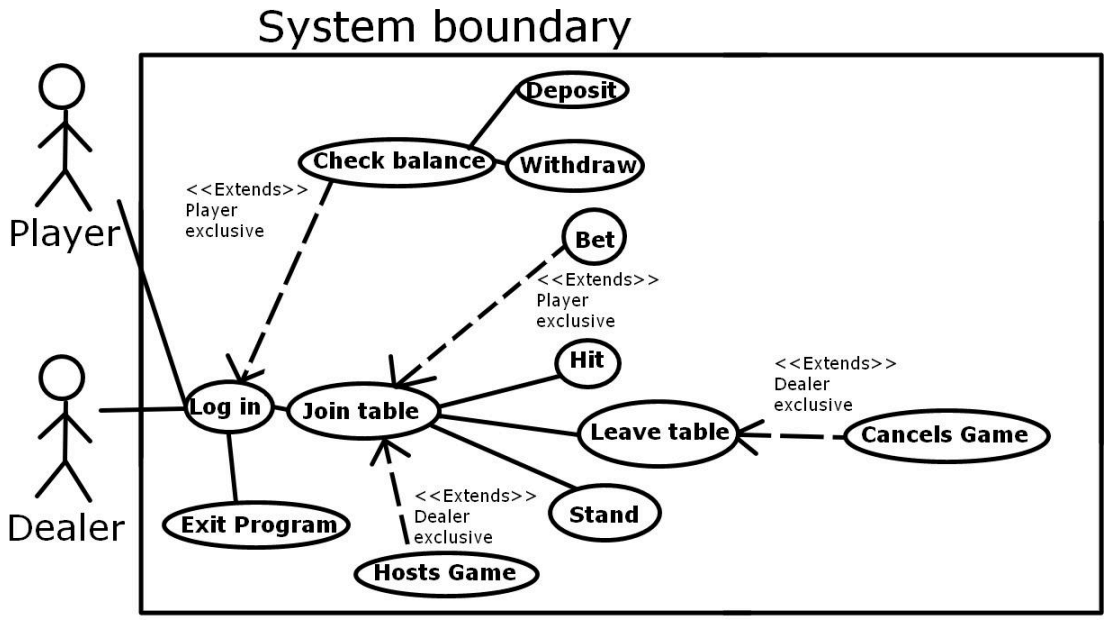
Use Case ID: DEALER-02
Use Case Name: “Hosts Game”
Relevant Requirements: 3.1.2, 3.1.3, 3.1.4, 3.1.5
Primary Actor: Dealer
Pre-conditions: Dealer is logged in,
Post-conditions: A new game of BlackJack becomes active and the server begins polling players for their bets
Basic Flow or Main Scenario:

1. The dealer selects the option to host/start the game from the Client UI
2. The Client sends a request to The Server to initiate the round
3. The Server calls the startGame() method within the Table class
4. The Table’s gameActive boolean attribute is set to true to lock the table state
5. The Server triggers the askForBets() method via the ClientHandler to prompt all players to input their wager

Related Use Cases:

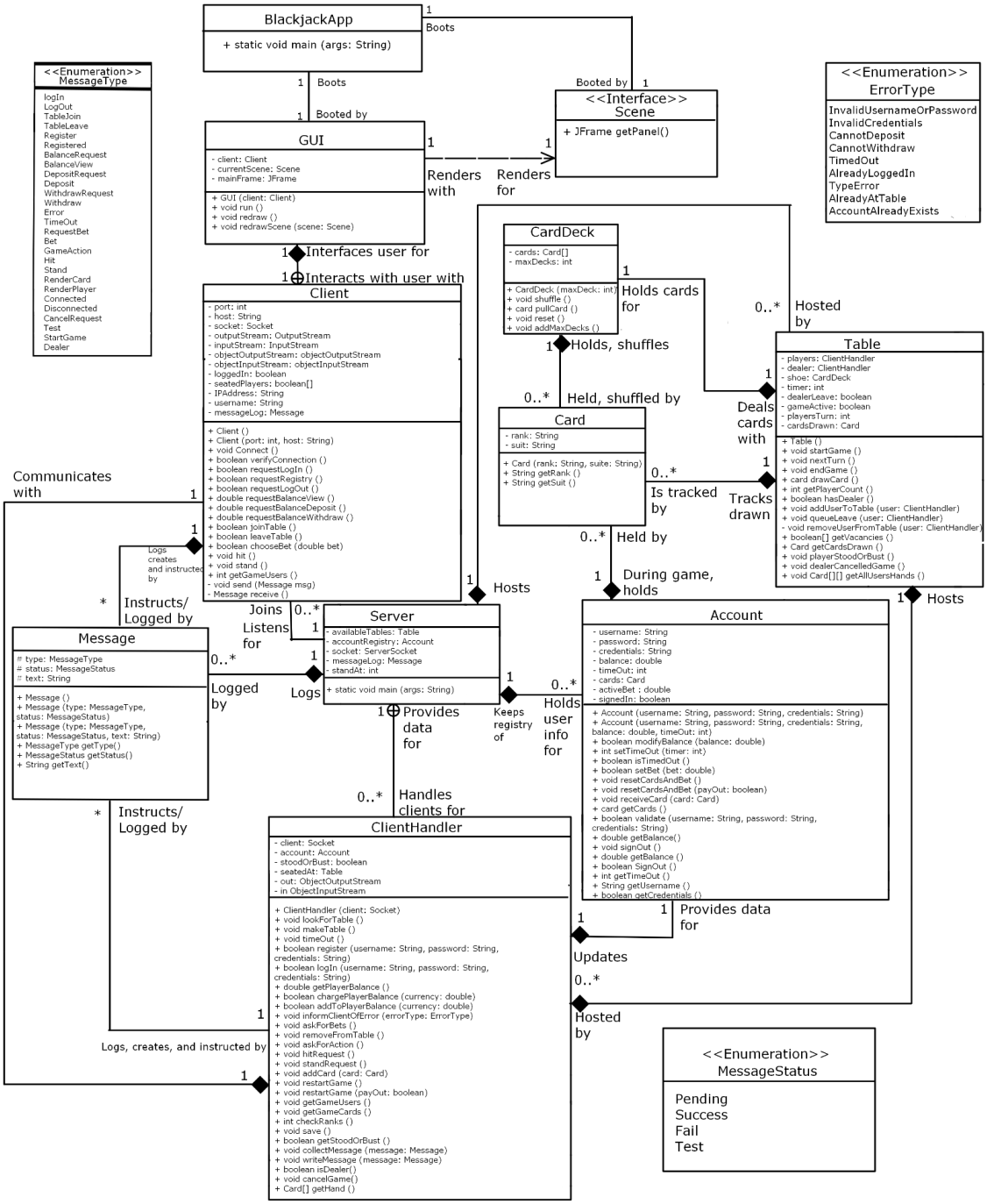
- USER-03: Log in
- PLAYER-05: Bet

UML Use Case Diagram

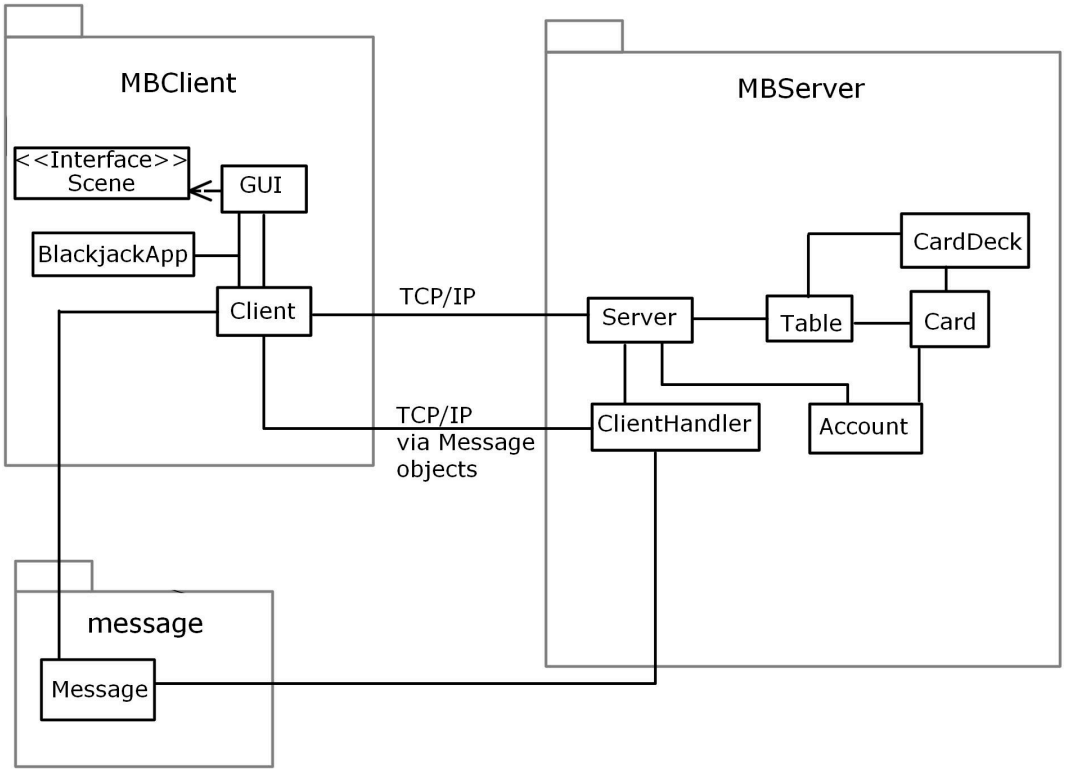


UML Class Diagrams

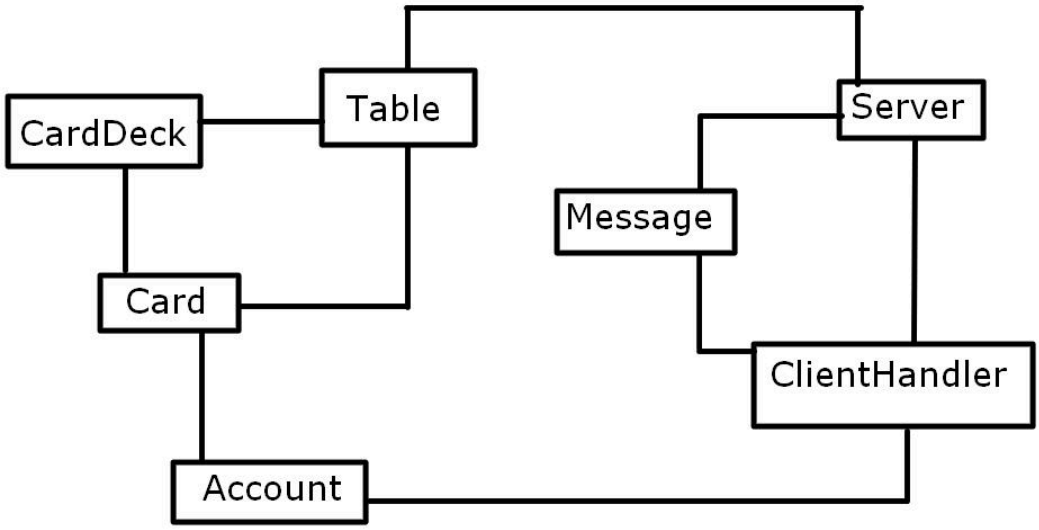
General UML Class Diagram:



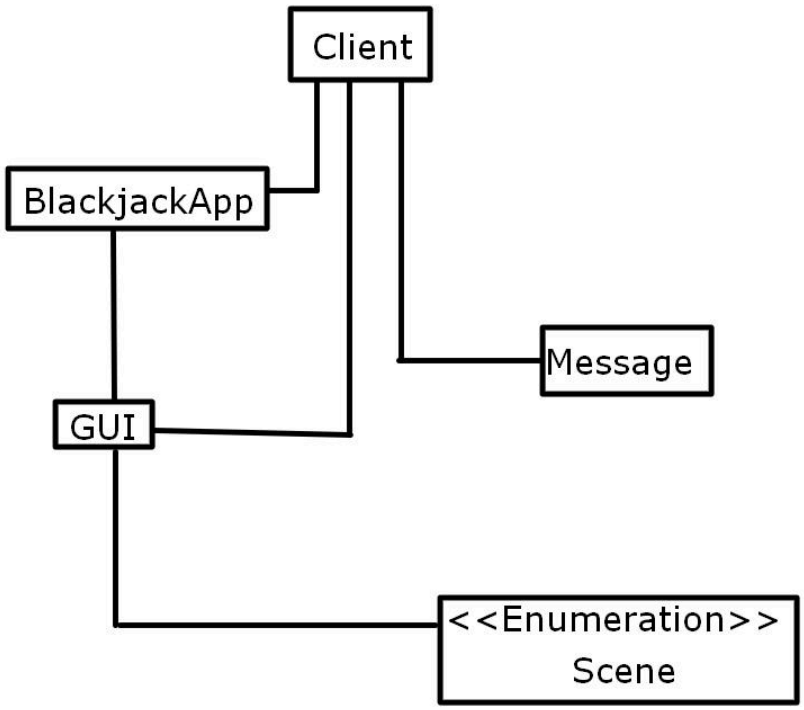
Simplified UML Class Diagram Displaying Packages:



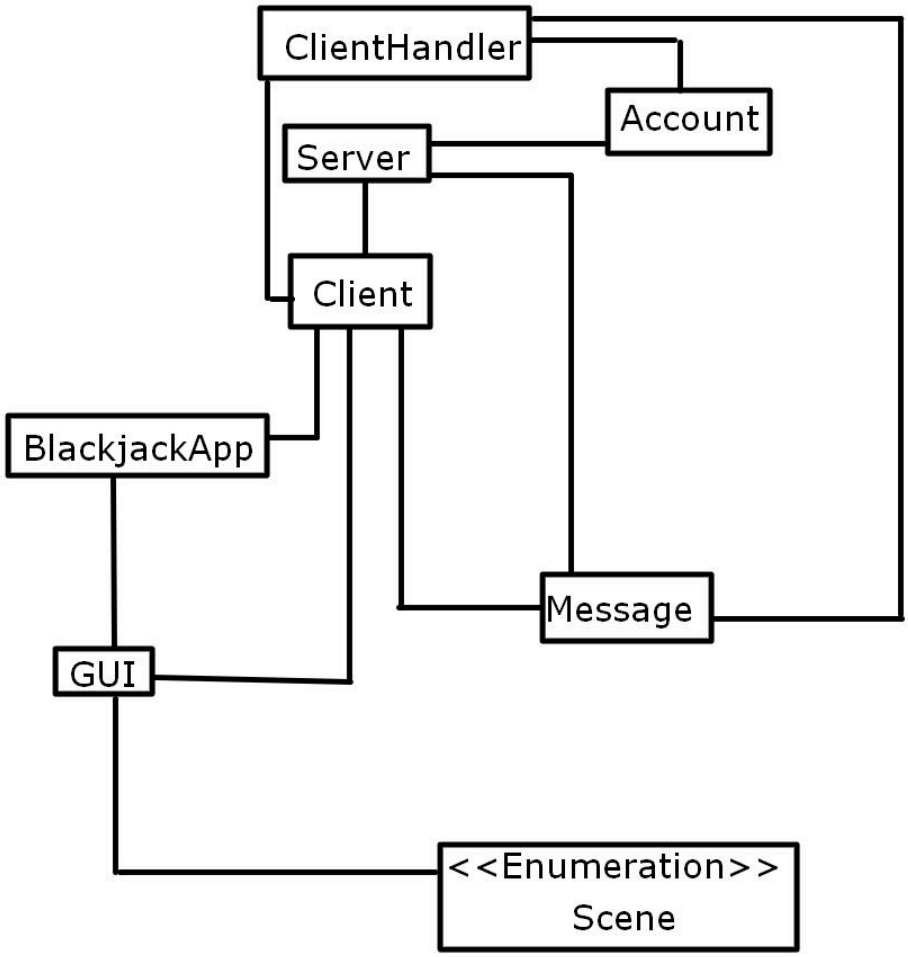
Server Module UML Class Diagram:



Client Module UML Class Diagram:



Dealer/Player Module UML Class Diagram:



Game Module UML Class Diagram:

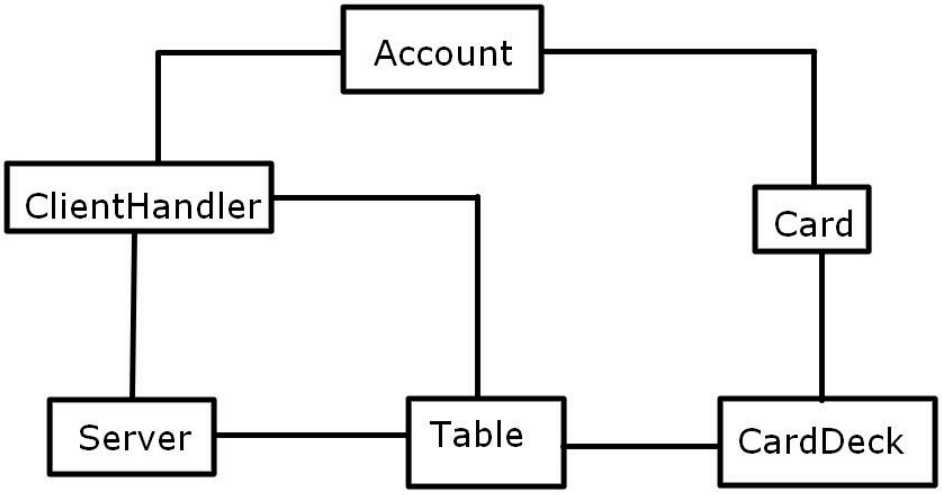
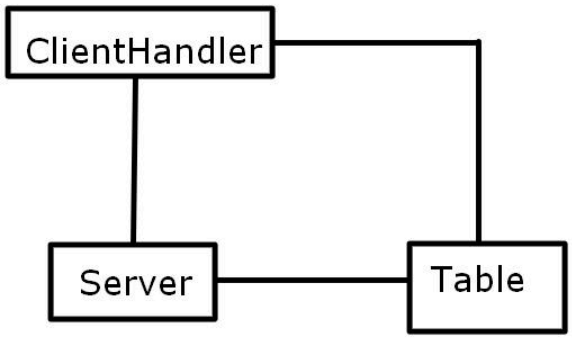
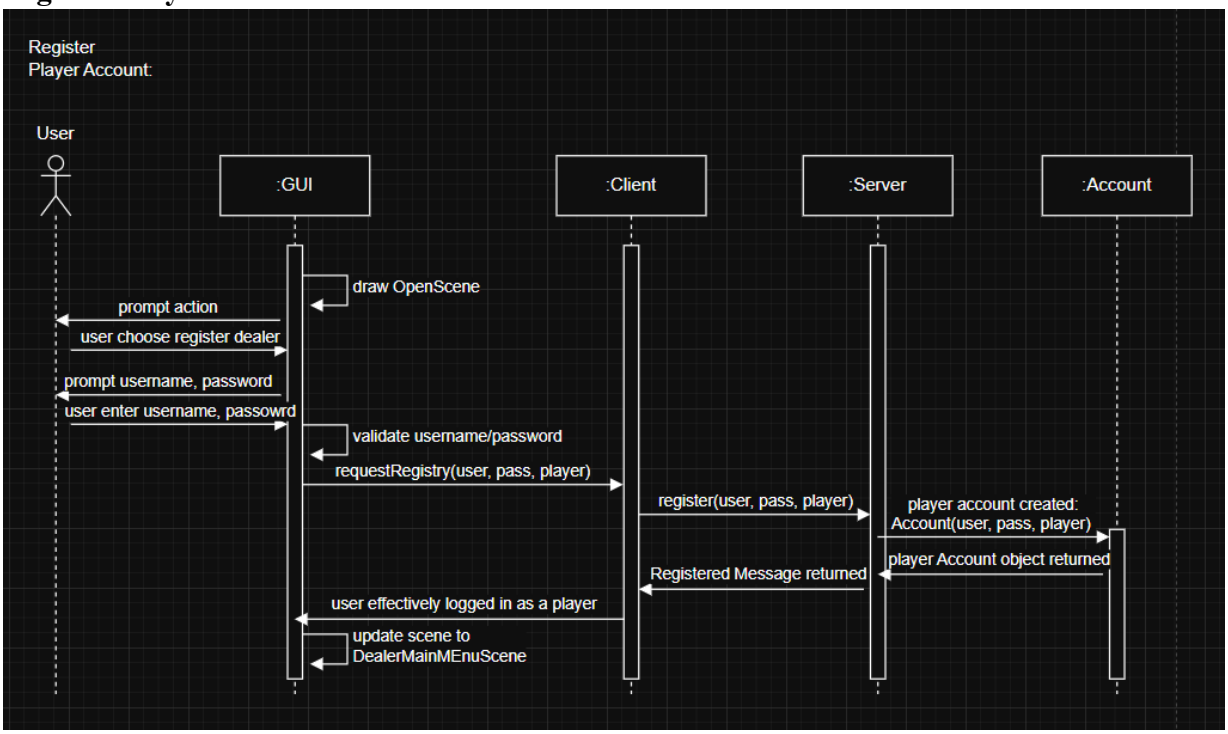


Table Module UML Class Diagram:

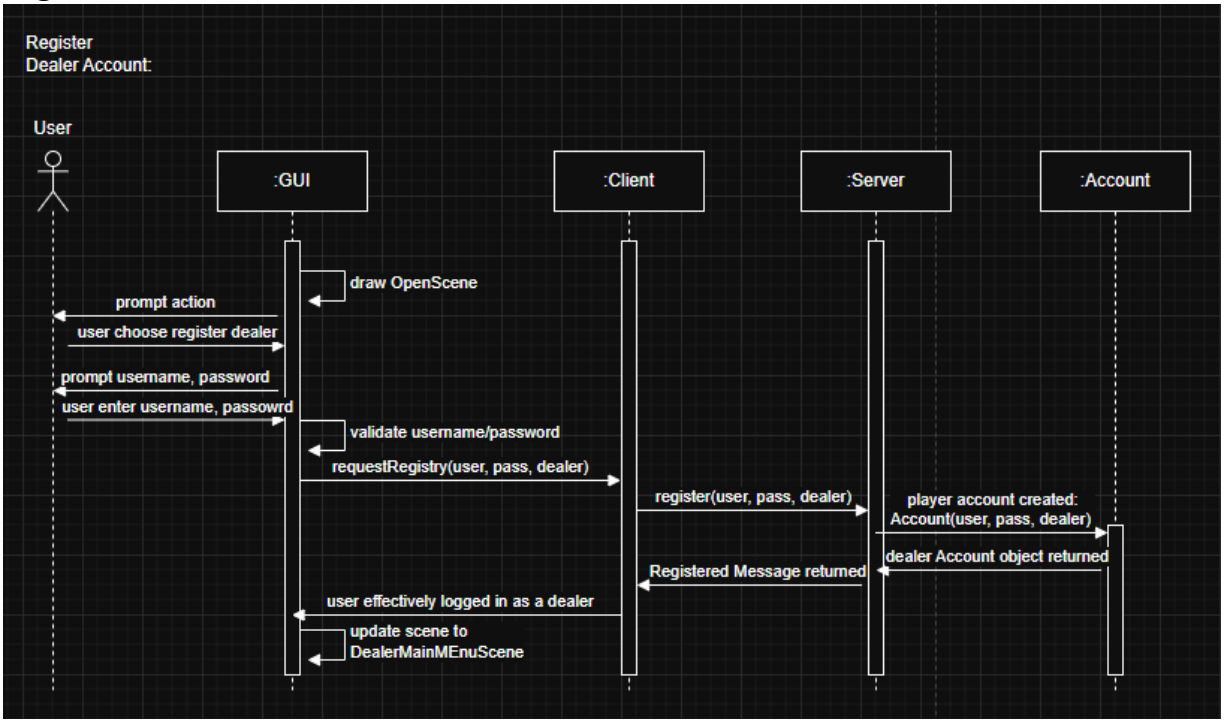


UML Sequence Diagrams

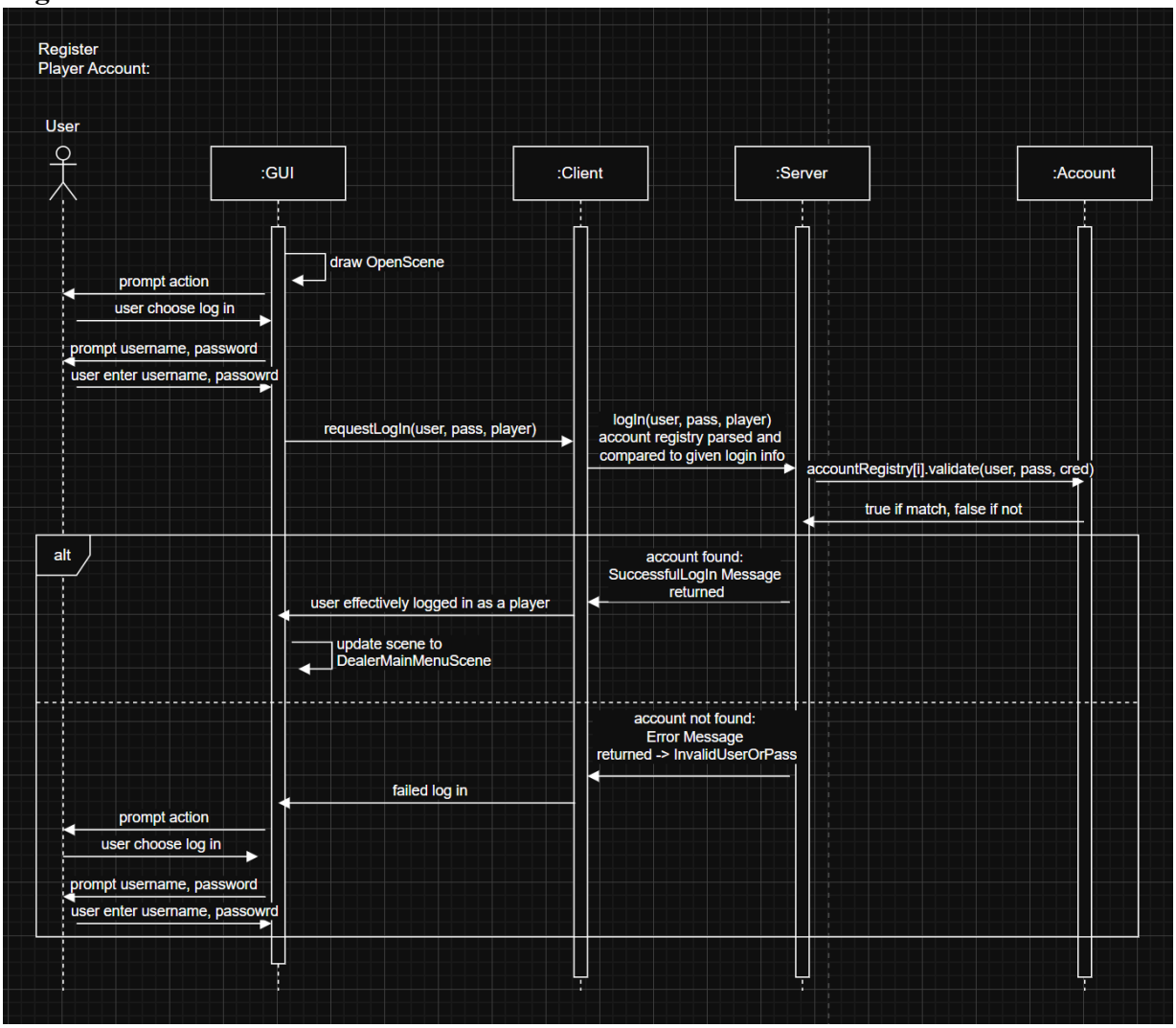
Register Player Account:



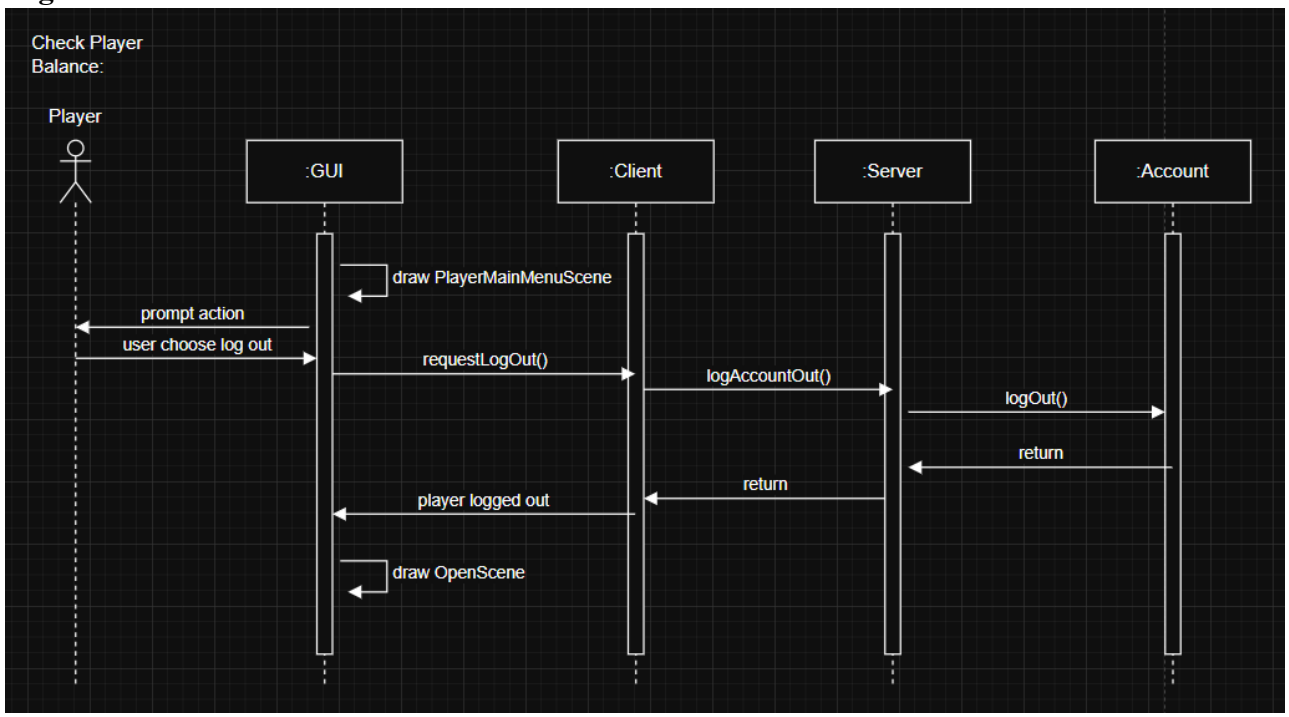
Register Dealer Account:



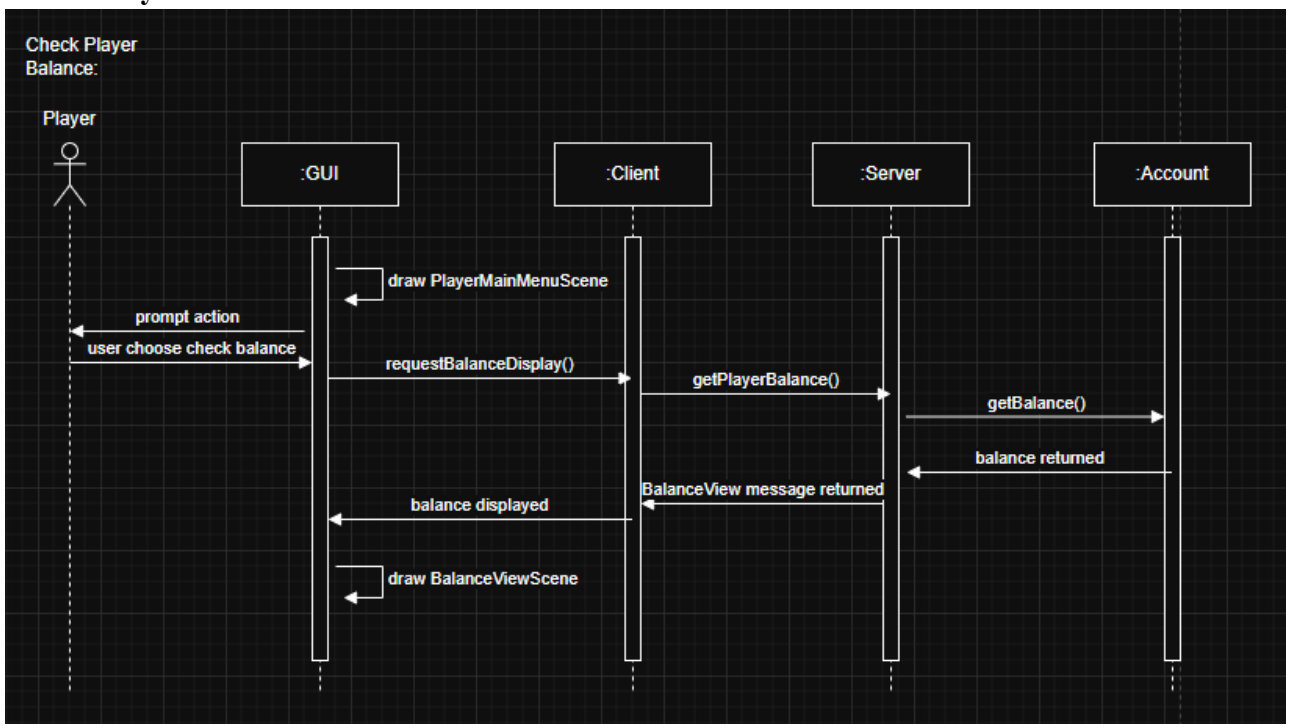
Log In


Register
Player Account:
User
:GUI
:Client
:Server
:Account
draw OpenScene
prompt action
user choose log in
prompt username, password
user enter username, passowrd
requestLogIn(user, pass, player)
logIn(user, pass, player)
account registry parsed and
compared to given login info
accountRegistry[i].validate(user, pass, cred)
true if match, false if not
alt
account found:
SuccessfulLogIn Message
returned
user effectively logged in as a player
update scene to
DealerMainMenuScene
account not found:
Error Message
returned -> InvalidUserOrPass
failed log in
prompt action
user choose log in
prompt username, password
user enter username, passowrd

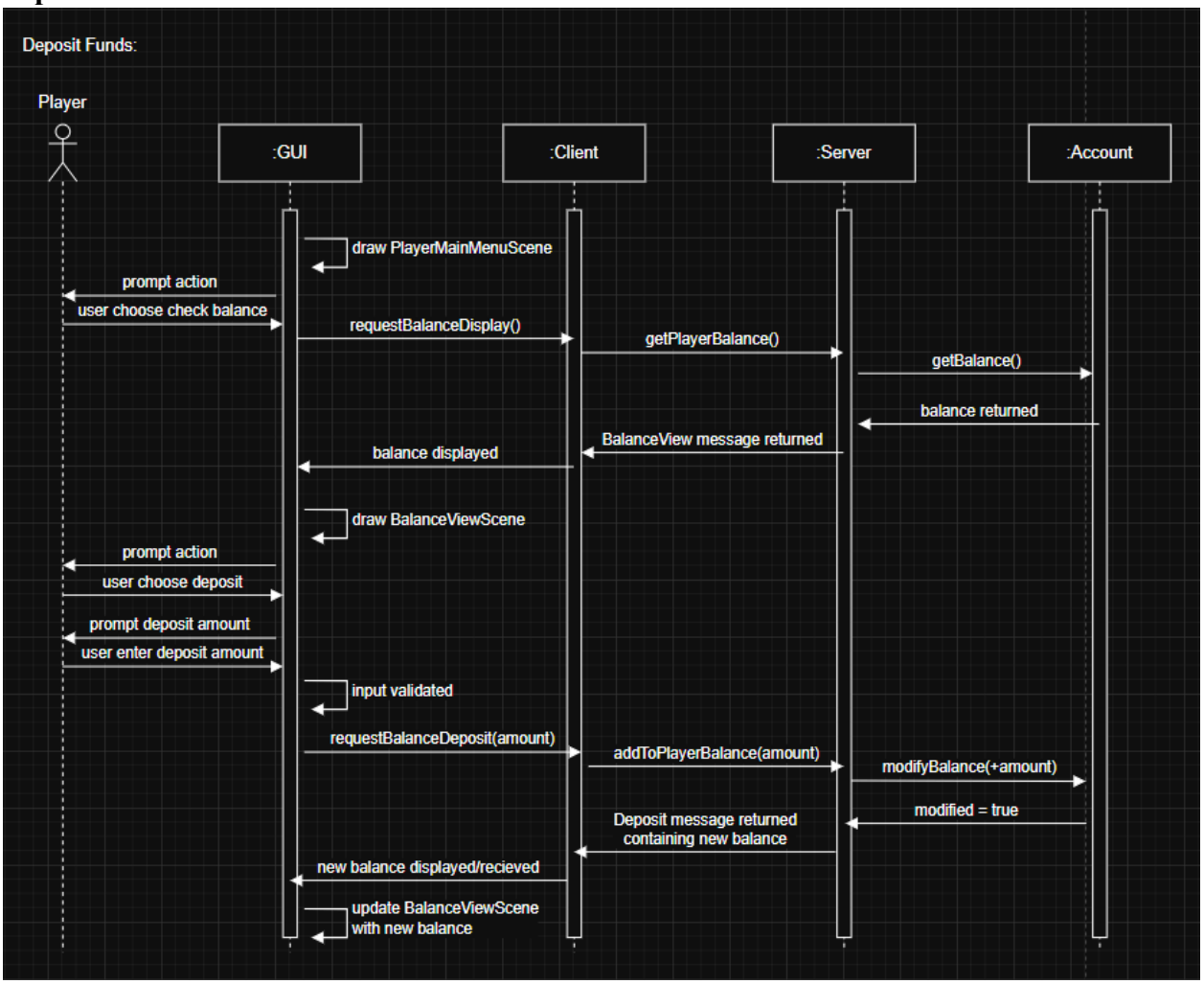
Log Out:



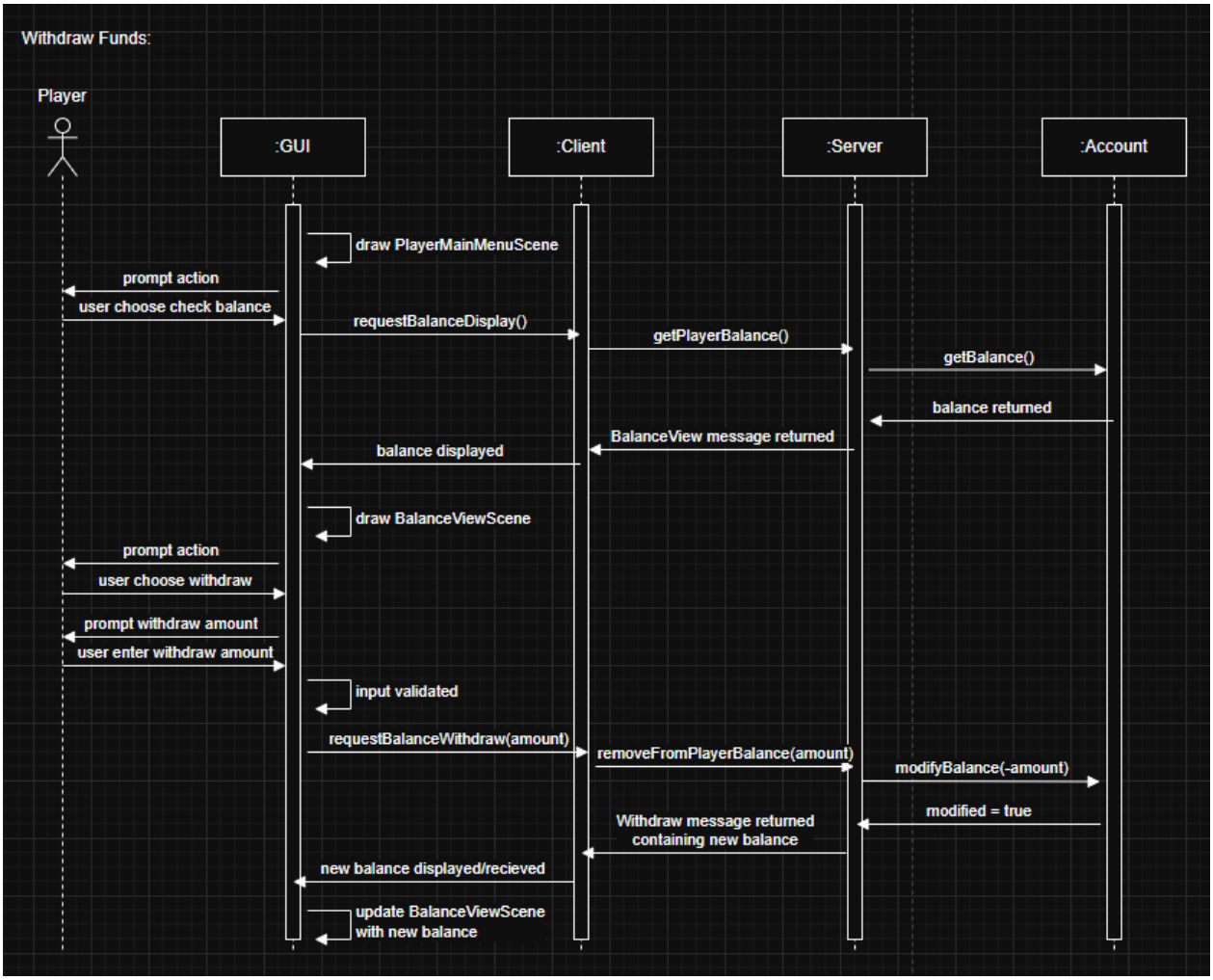
Check Player Balance:



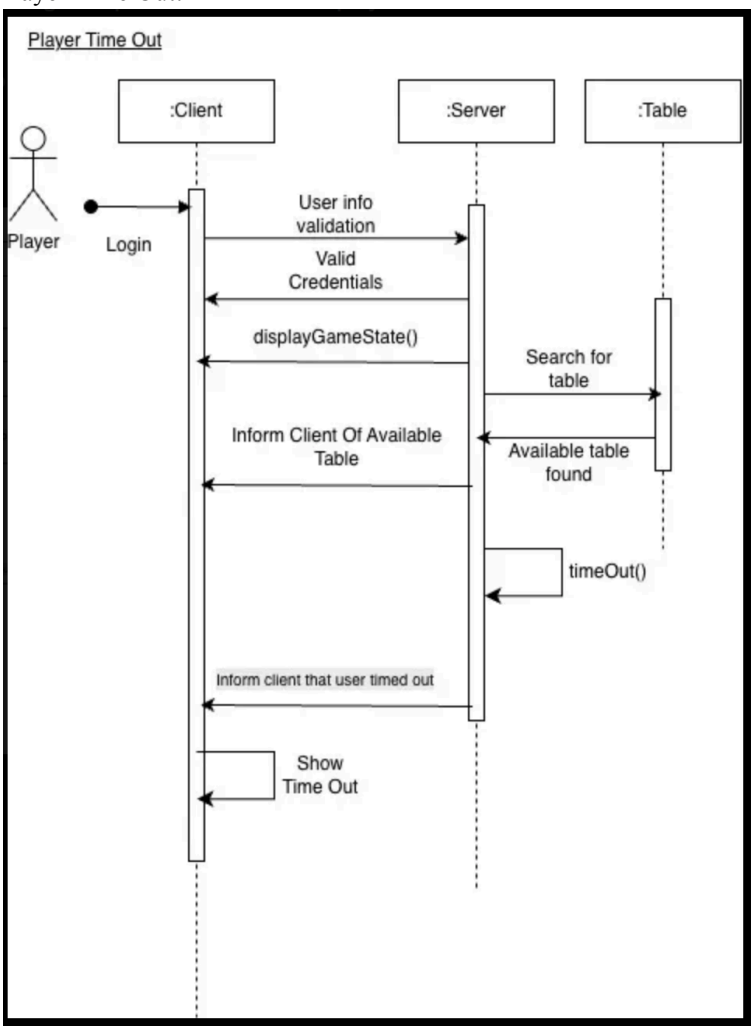
Deposit Funds:



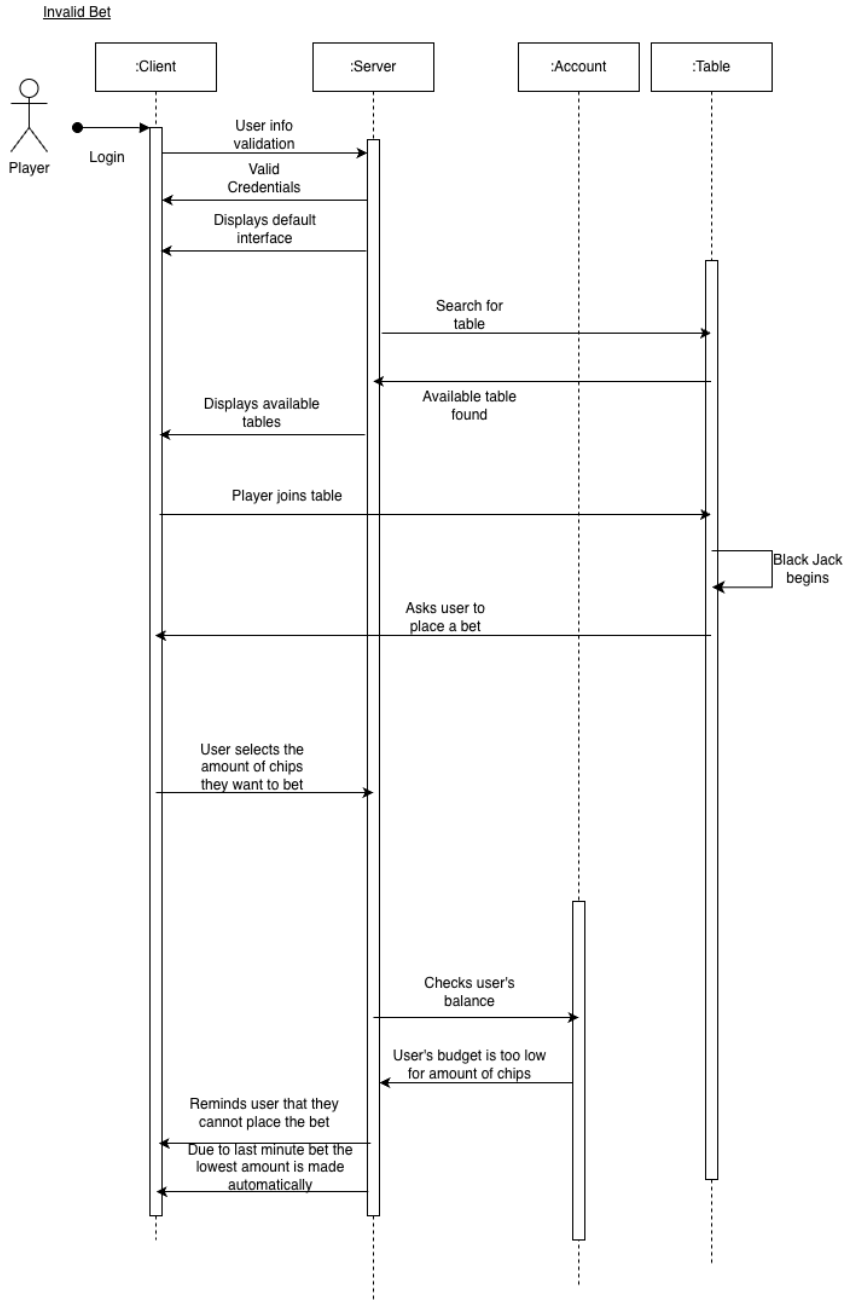
Withdraw Funds:



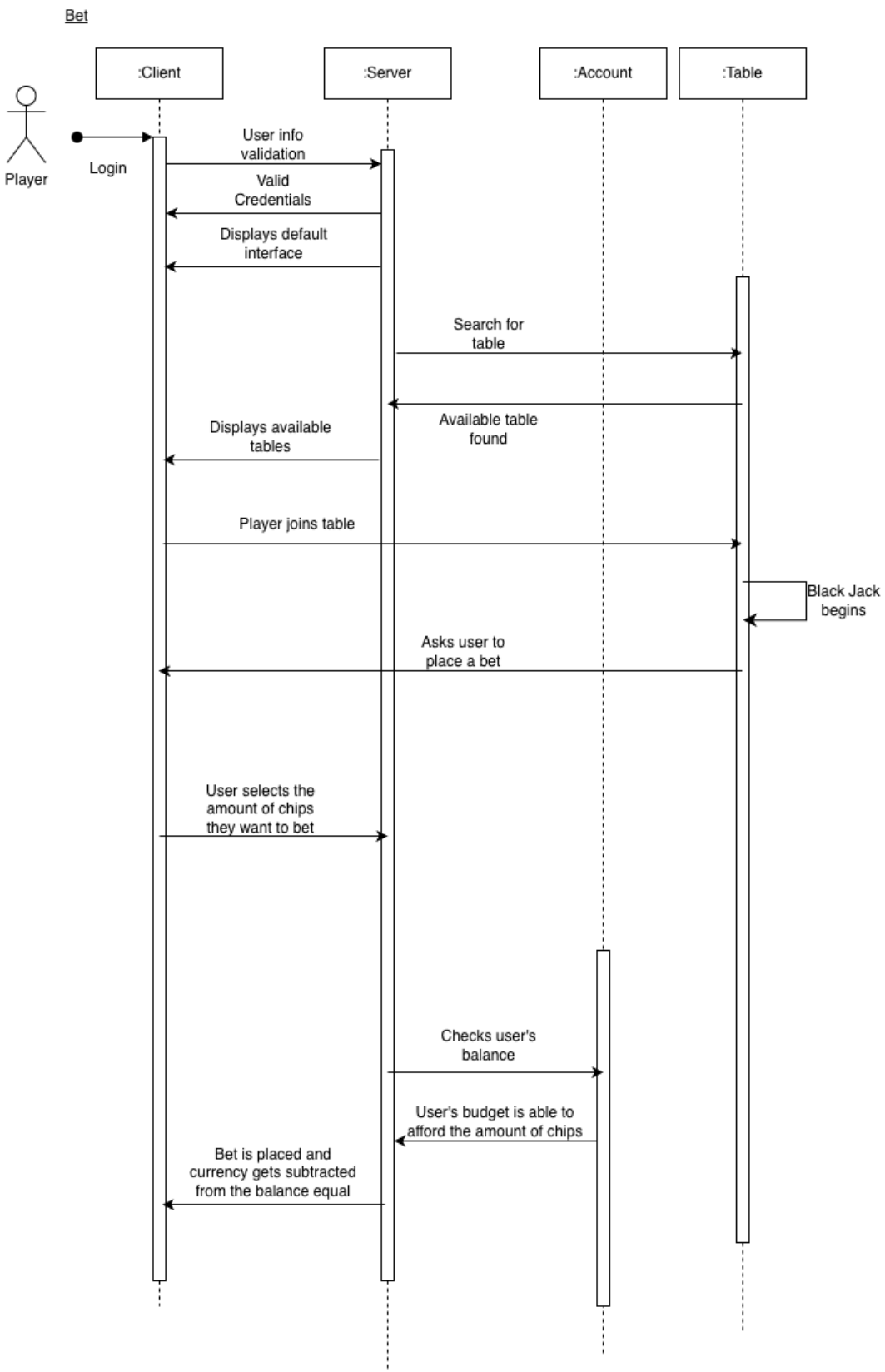
Player Time Out:



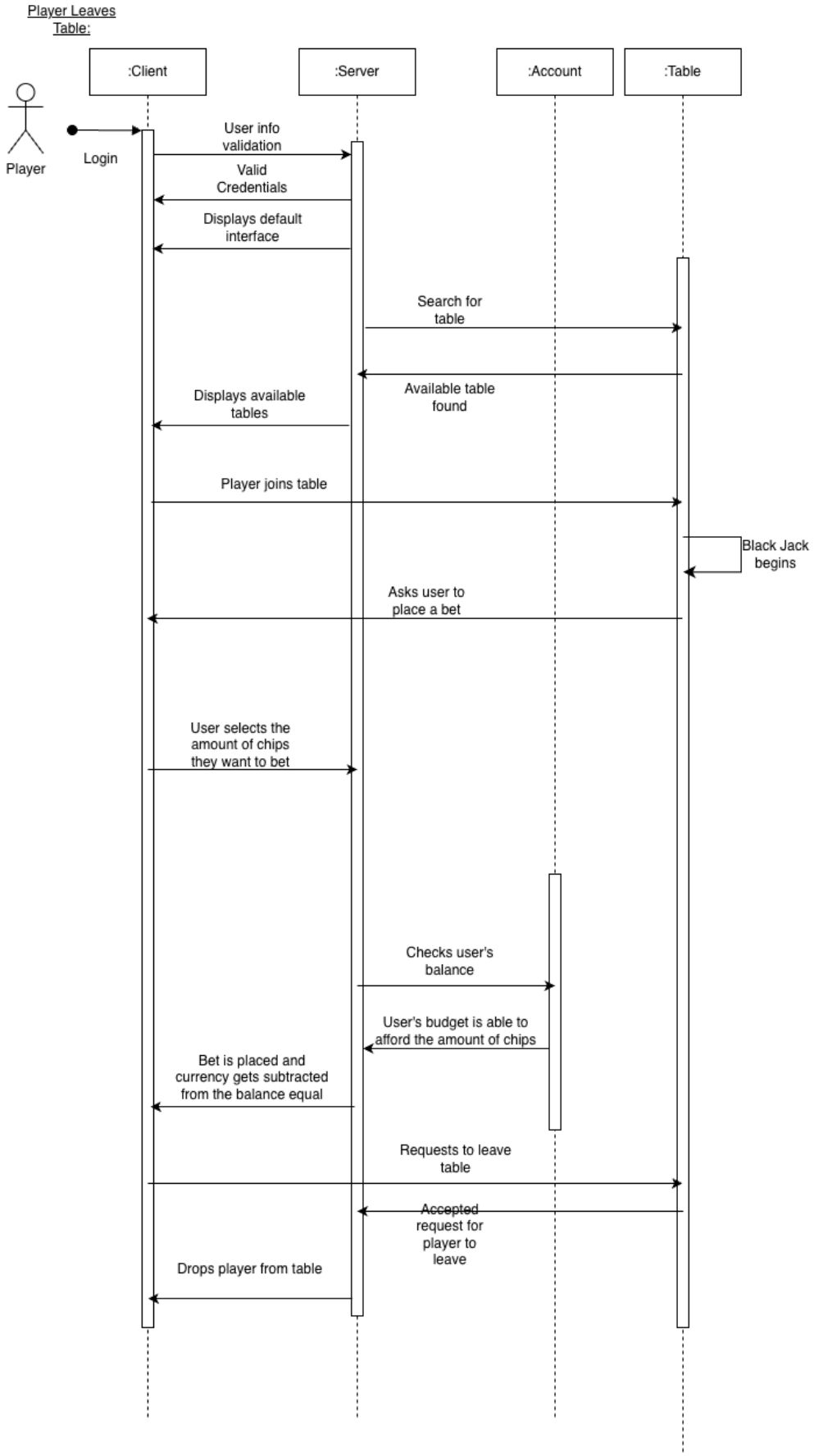
Player makes an invalid Bet:



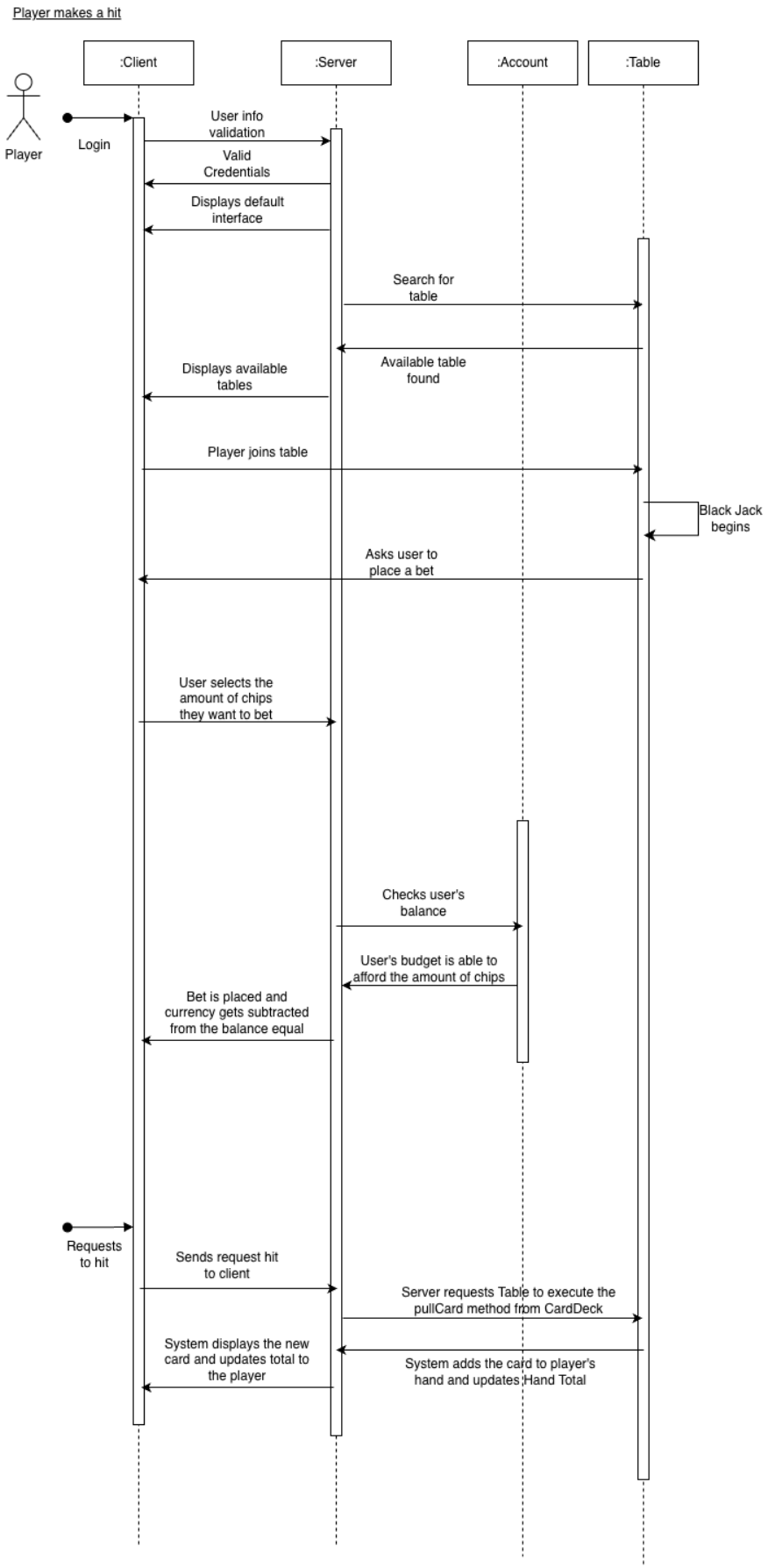
Bet:


Bet
:Client
:Server
:Account
:Table
Player
Login
User info validation
Valid Credentials
Displays default interface
Search for table
Available table found
Displays available tables
Player joins table
Black Jack begins
Asks user to place a bet
User selects the amount of chips they want to bet
Checks user's balance
User's budget is able to afford the amount of chips
Bet is placed and currency gets subtracted from the balance equal

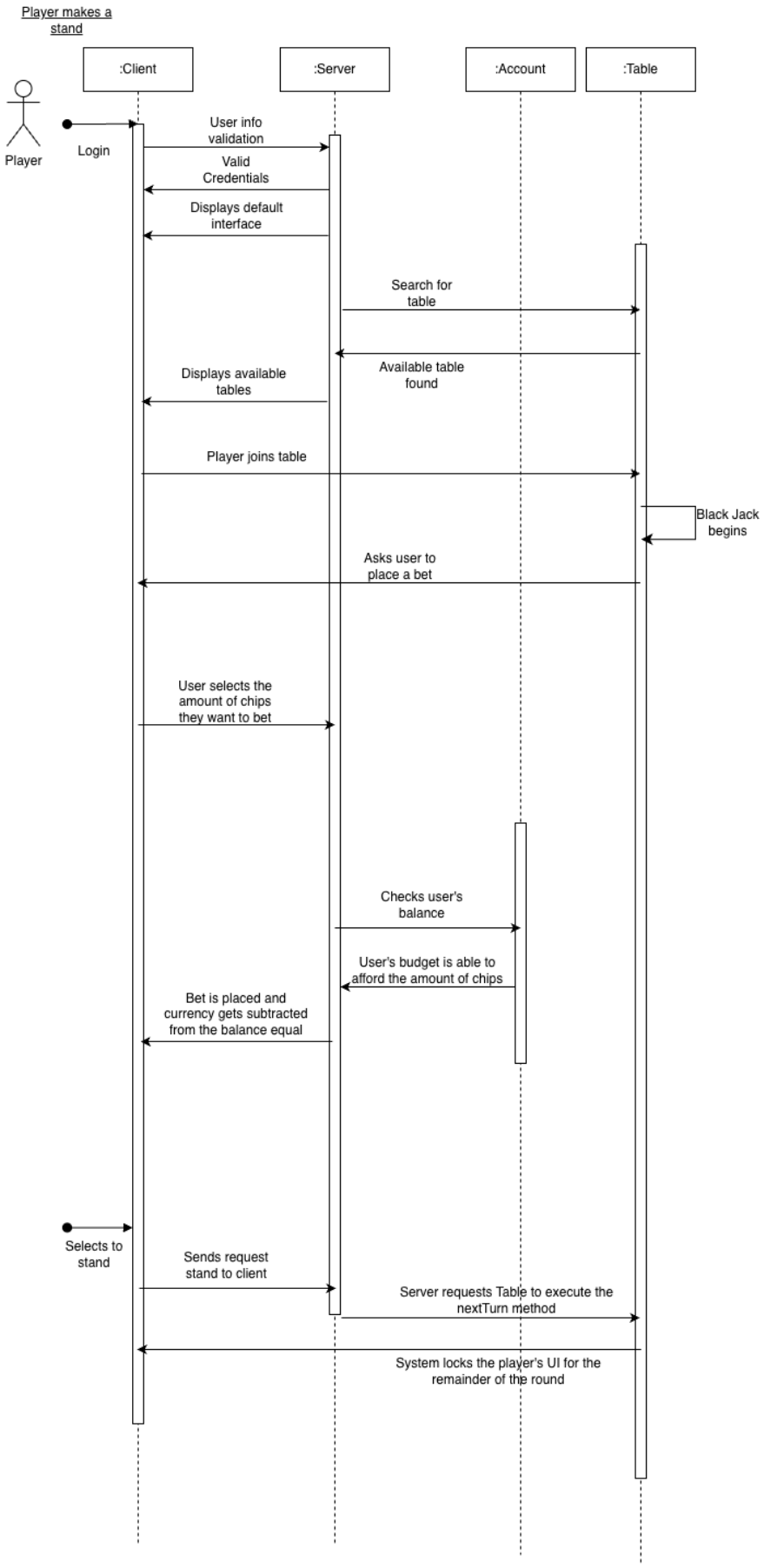
Player Leaves Table:


Player Leaves Table:
:Client
:Server
:Account
:Table
Player
Login
User info validation
Valid Credentials
Displays default interface
Search for table
Available table found
Displays available tables
Player joins table
Black Jack begins
Asks user to place a bet
User selects the amount of chips they want to bet
Checks user's balance
User's budget is able to afford the amount of chips
Bet is placed and currency gets subtracted from the balance equal
Requests to leave table
Accepted request for player to leave
Drops player from table

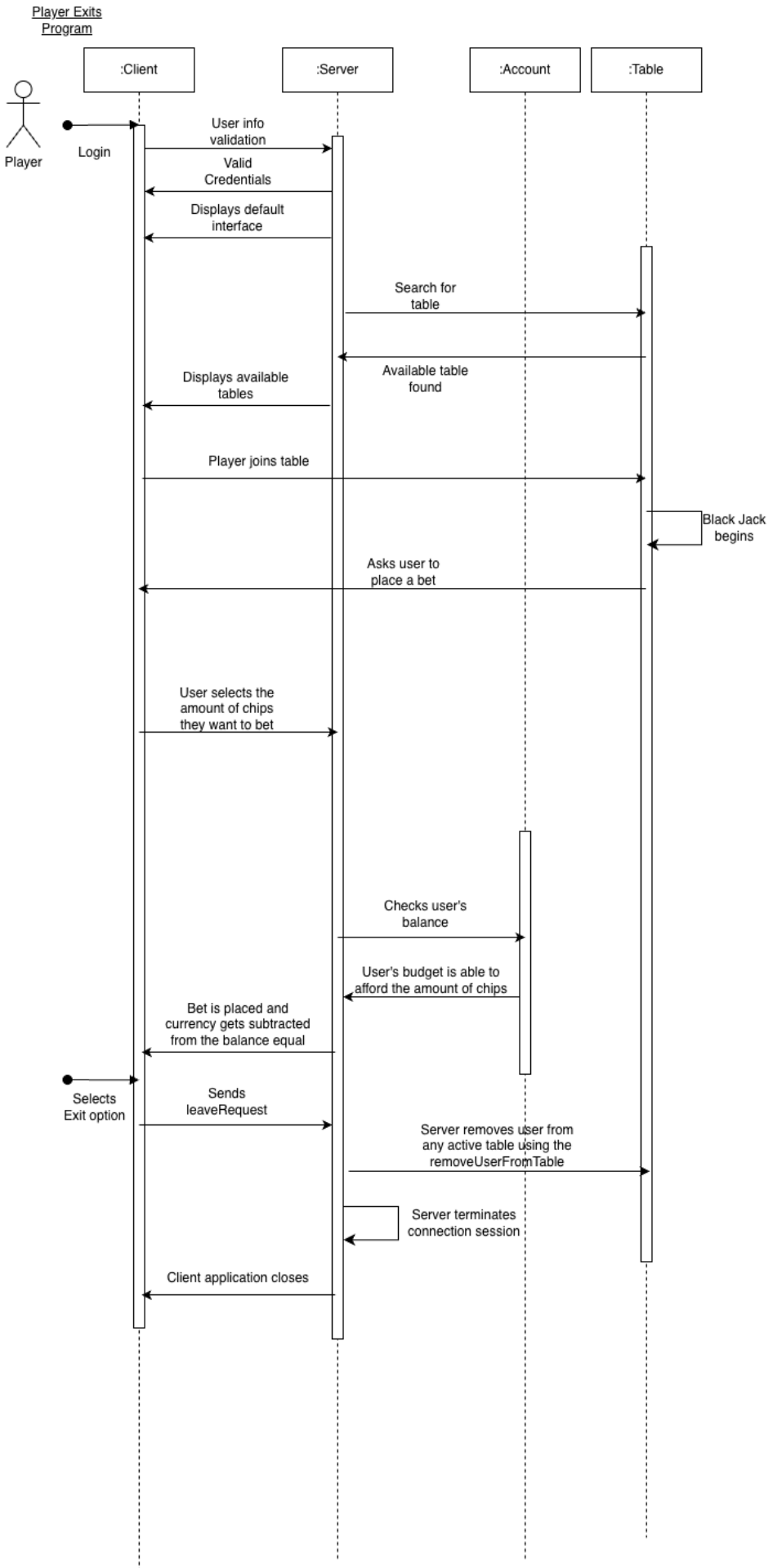
Players makes a hit:


Player makes a hit
:Client
:Server
:Account
:Table
Player
Login
User info validation
Valid Credentials
Displays default interface
Search for table
Available table found
Displays available tables
Player joins table
Black Jack begins
Asks user to place a bet
User selects the amount of chips they want to bet
Checks user's balance
User's budget is able to afford the amount of chips
Bet is placed and currency gets subtracted from the balance equal
Requests to hit
Sends request hit to client
Server requests Table to execute the pullCard method from CardDeck
System displays the new card and updates total to the player
System adds the card to player's hand and updates Hand Total

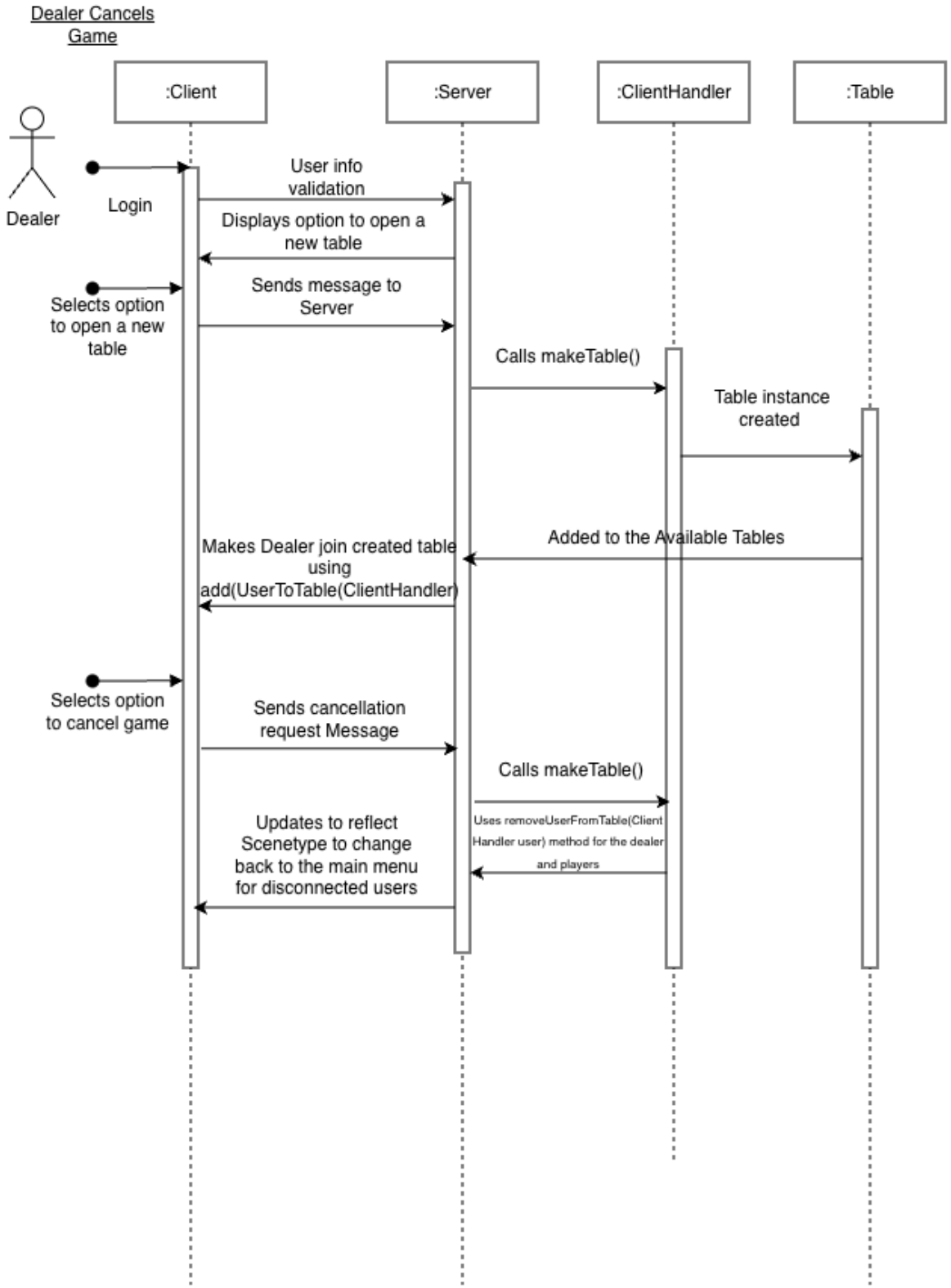
Player stands:


Player makes a stand
:Client
:Server
:Account
:Table
Player
Login
User info validation
Valid Credentials
Displays default interface
Search for table
Available table found
Displays available tables
Player joins table
Black Jack begins
Asks user to place a bet
User selects the amount of chips they want to bet
Checks user's balance
User's budget is able to afford the amount of chips
Bet is placed and currency gets subtracted from the balance equal
Selects to stand
Sends request stand to client
Server requests Table to execute the nextTurn method
System locks the player's UI for the remainder of the round

Player Exits Program:


Player Exits Program
:Client
:Server
:Account
:Table
Player
Login
User info validation
Valid Credentials
Displays default interface
Search for table
Available table found
Displays available tables
Player joins table
Black Jack begins
Asks user to place a bet
User selects the amount of chips they want to bet
Checks user's balance
User's budget is able to afford the amount of chips
Bet is placed and currency gets subtracted from the balance equal
Selects Exit option
Sends leaveRequest
Server removes user from any active table using the removeUserFromTable
Server terminates connection session
Client application closes

Dealer Cancels Game:



Dealer Hosts Game:

Dealer Hosts Game